

國立高雄科技大學
National Kaohsiung University of Science and Technology

深度學習理論與實作
CH5 電腦視覺的深度學習

資工系 陳俊豪 教授



Outline



5-1 卷積神經網路 CNN

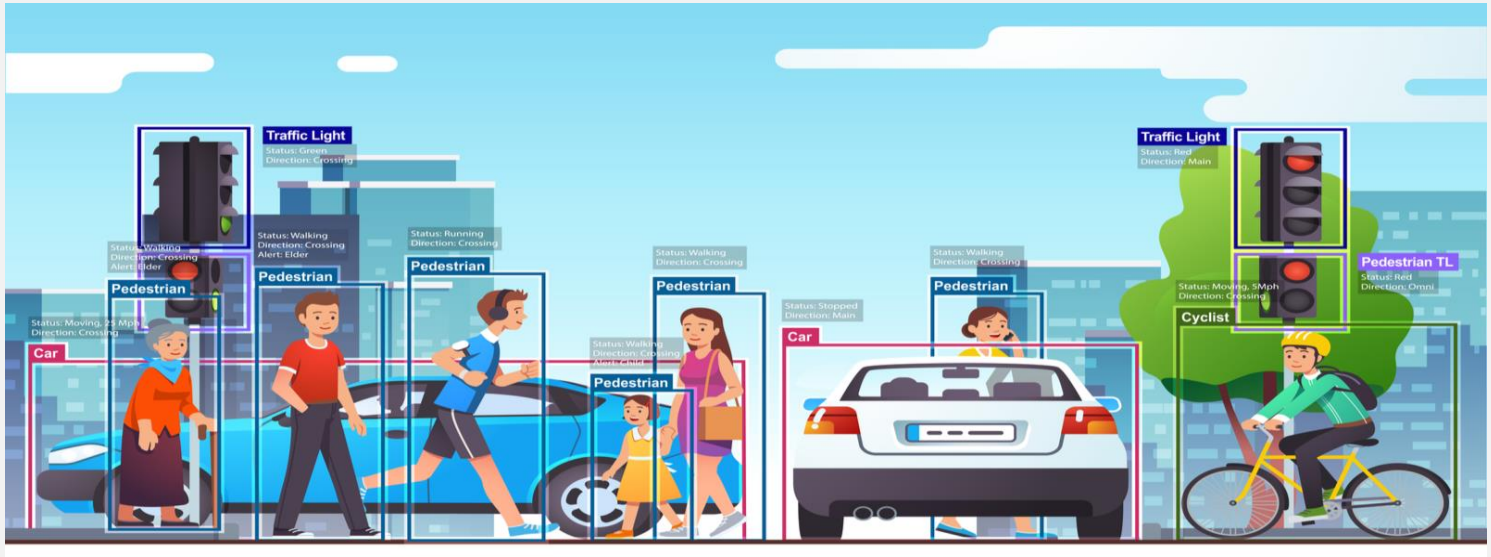
5-2 以少量資料集從頭訓練一個卷積神經網路

5-3 使用預先訓練的卷積神經網路

5-4 視覺化呈現卷積神經網路學習的內容

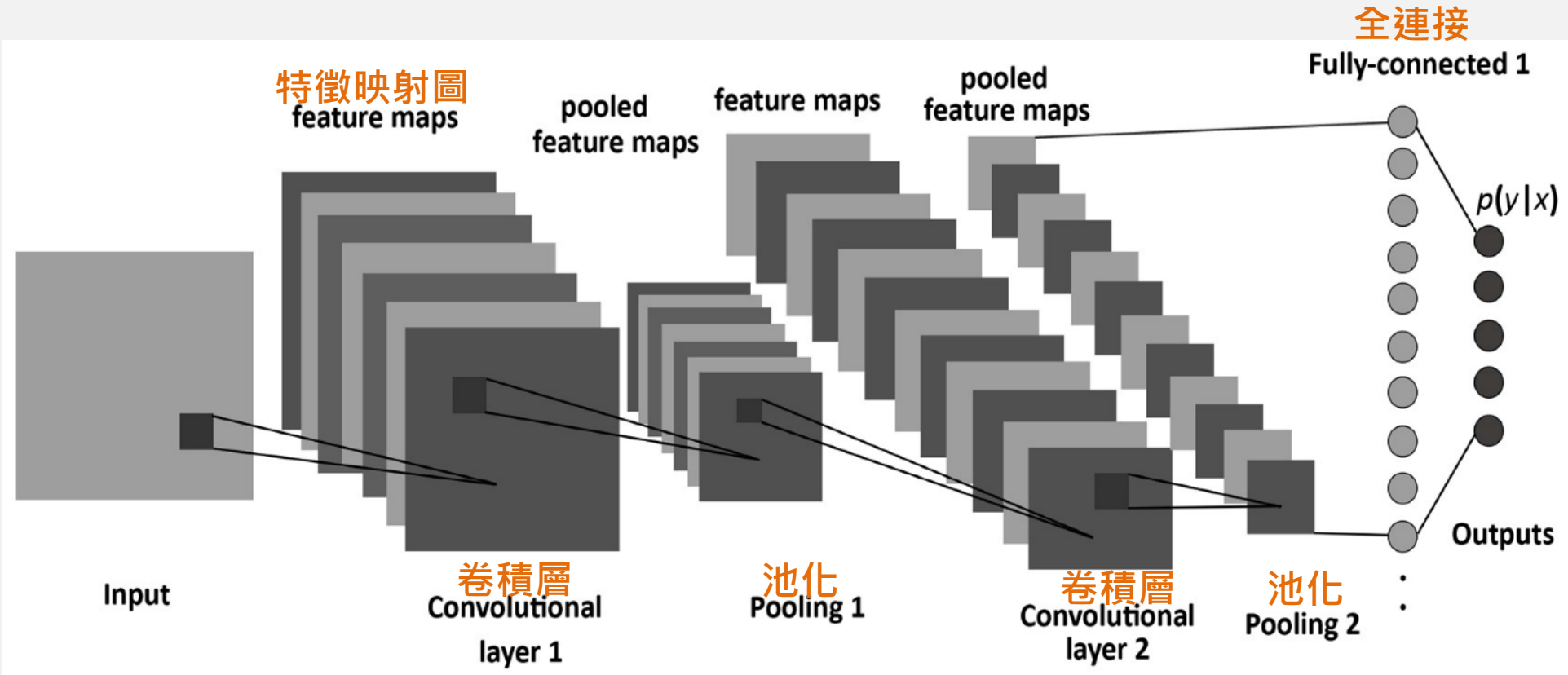
5-1 卷積神經網路 CNN

- ✓ 卷積神經網路(convolutional neural networks, CNN)
  - 普遍用於**電腦視覺**的深度學習模型
  - 電腦視覺Computer Vision
    - ✓ 「訓練」電腦看懂圖片和影片的研究領域，電腦能看懂影像，也就是能處理圖像數據，從圖片或影片中擷取資料，應用領域如：自動駕駛功能等



5-1 卷積神經網路 CNN

✓ 卷積神經網路(CNN)普遍用於電腦視覺的**深度學習模型**



5-1 卷積神經網路 CNN

✓ 先實際看一個簡單的卷積神經網路，用CNN對MNIST數字進行分類

- 初始化一個小型卷積神經網路：3個Conv2D層(2D卷積層)和2個MaxPooling2D層(最大池化層)
- Param(參數): 神經網路的權重 (= 過濾器個數*過濾器長*寬*輸入通道數 +過濾器個數)

```
from keras import layers  # 從 keras 套件匯入 layers, models 套件
from keras import models

model = models.Sequential()
                    #過濾器數量 ↓     ↓過濾器長寬
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))# 加入 Covn2d 層
model.add(layers.MaxPooling2D((2, 2)))   # 加入 MaxPooling 2D層
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))

model.summary()
```

5-1 卷積神經網路 CNN

✓ 神經網路架構之Summary，此例共55744個參數需要學習

Out[] :

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 26, 26, 32)        320
 max_pooling2d (MaxPooling2D  (None, 13, 13, 32)       0
 )
 conv2d_1 (Conv2D)           (None, 11, 11, 64)        18496
 max_pooling2d_1 (MaxPooling  (None, 5, 5, 64)         0
 2D)
 conv2d_2 (Conv2D)           (None, 3, 3, 64)          36928
=================================================================
Total params: 55,744
Trainable params: 55,744
Non-trainable params: 0
_________________________________________________________________
```

(= 32 * 3 * 3 + 32)

(= (64 * 3 * 3)*32 + 64)

(= (64 * 3 * 3)*64 + 64)

5-1 卷積神經網路 CNN

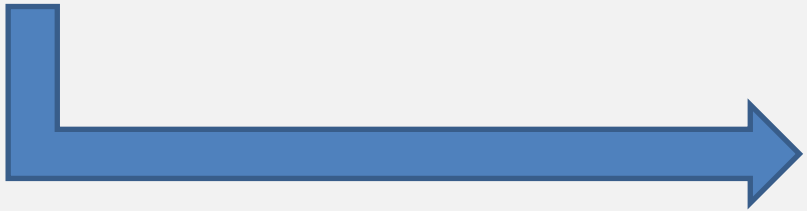
✓ 在卷積神經網路上加入分類器

- 分類器神經網路能處理的是1D向量，因此需將3D張量轉1D向量

```
model.add(layers.Flatten())  # 將 3D 張量展開攤平為 1D, 其 shape = (576, )
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

model.summary()
```

(576 = 64 * 3 * 3)



現在神經網路的架構：

(36928 = 576 * 64 + 64)

(650 = 64 * 10 + 10)

Out[]：

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_3 (Conv2D)            (None, 26, 26, 32)        320

max_pooling2d_2 (MaxPooling  (None, 13, 13, 32)        0
2D)

conv2d_4 (Conv2D)            (None, 11, 11, 64)        18496

max_pooling2d_3 (MaxPooling  (None, 5, 5, 64)          0
2D)

conv2d_5 (Conv2D)            (None, 3, 3, 64)          36928

flatten (Flatten)            (None, 576)               0

dense (Dense)                (None, 64)                36928

dense_1 (Dense)              (None, 10)                650

=================================================================
Total params: 93,322
Trainable params: 93,322
Non-trainable params: 0
_________________________________________________________________
```

93,322

5-1 卷積神經網路 CNN



✓ 用 MNIST 影像訓練卷積神經網路

```
from keras.datasets import mnist
from keras.utils import to_categorical

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28, 28, 1))
train_images = train_images.astype('float32') / 255
test_images = test_images.reshape((10000, 28, 28, 1))
test_images = test_images.astype('float32') / 255

train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)
#編譯與訓練模型
model.compile(optimizer='rmsprop' , loss='categorical_crossentropy',metrics=['accuracy'])
model.fit(train_images, train_labels, epochs=5, batch_size=64)
```

5-1 卷積神經網路 CNN

✓ 用 MNIST 影像訓練卷積神經網路結果

Out[] :

```
Epoch 1/5
938/938 [==============================] - 20s 20ms/step - loss: 0.1801 - accuracy: 0.9431
Epoch 2/5
938/938 [==============================] - 20s 21ms/step - loss: 0.0495 - accuracy: 0.9847
Epoch 3/5
938/938 [==============================] - 20s 22ms/step - loss: 0.0330 - accuracy: 0.9893
Epoch 4/5
938/938 [==============================] - 20s 22ms/step - loss: 0.0243 - accuracy: 0.9928
Epoch 5/5
938/938 [==============================] - 20s 22ms/step - loss: 0.0199 - accuracy: 0.9940
```

與第二章使用密集連接(Dense)神經網路做比較(當時測試的精準度為97.8%)，此次結果為99.40%。
即使使用簡單的CNN，**精準度也超越密集連接的神經網路**

5-1 卷積神經網路 CNN



✓ 用測試資料來評估 model

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
Print(test_acc)
```

Out[] :

```
313/313 [==============================] - 2s 6ms/step - loss: 0.0279 - accuracy: 0.9917

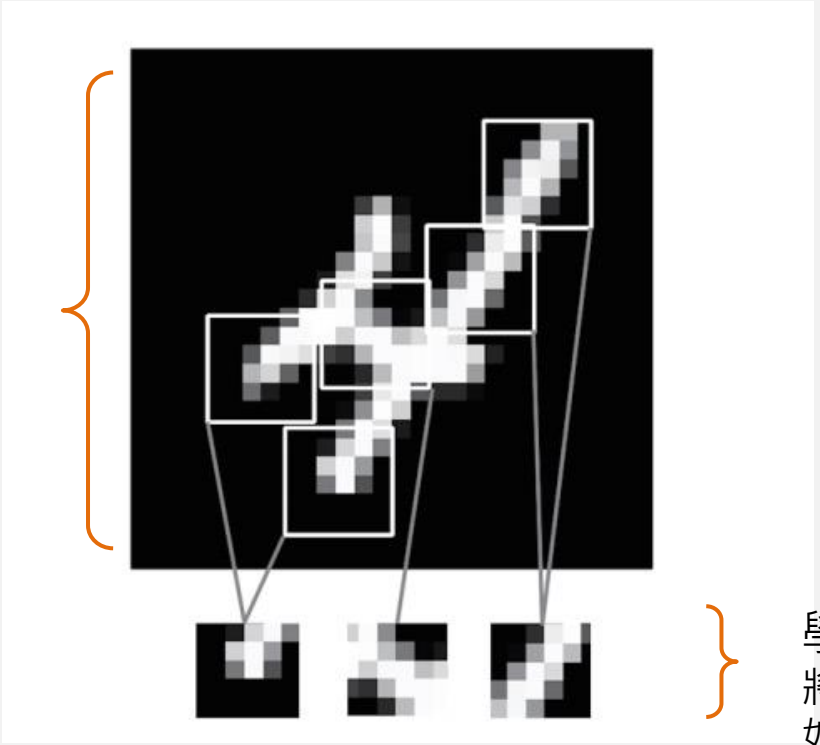
0.9916999936103821
```

5-1-1 卷積層

✓ 密集連接層和卷積層之間的區別：

密集層

會由輸入的特徵空間中
學習**全域的pattern**



卷積層

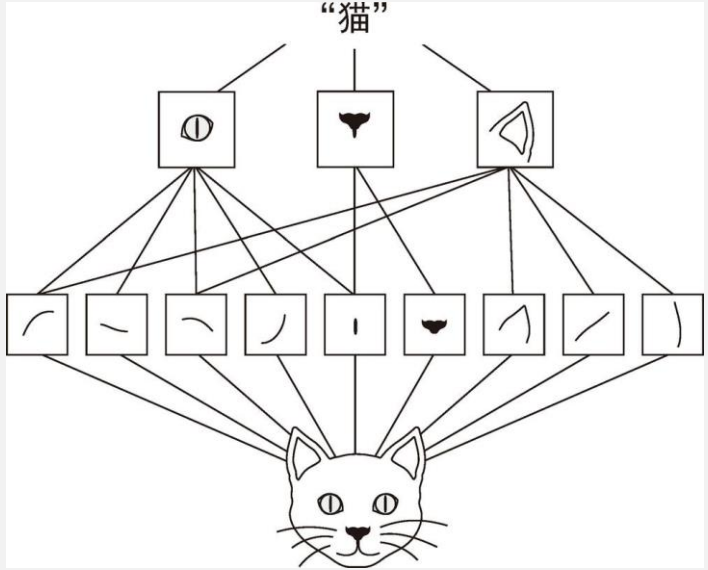
學習**局部的pattern**
將影像分解為局部圖案，
如：邊緣、紋理等

5-1-1 卷積層 - CNN特性

✓ 卷積神經網路的特性

- 學習到的pattern具**平移不變性**(translation invariant)
  - ✓ 可在較少的訓練樣本下找到共通性
  - ✓ 如：在影像的右下角學習到的pattern，卷積神經網路可以在任何位置識別該pattern
- 學習到pattern的**空間層次結構**(spatial hierarchies of patterns)
  - ✓ 第一卷積層會學到局部圖案
  - ✓ 第二卷積層會基於第一層來學習

#卷積神經網路：
將人類視覺所建的圖案，
拆分為局部圖案，
再組合成高階抽象事物
(如：貓)

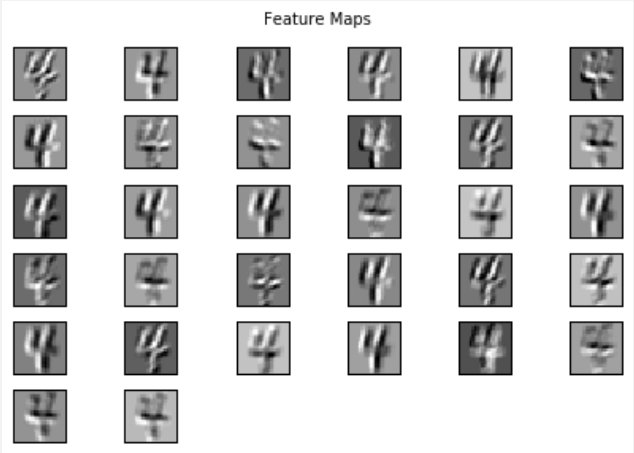


5-1-1 卷積層 - 特徵圖

✓ 特徵映射圖(feature maps)，即特徵圖 ➔ 卷積神經網路所運用的3D張量

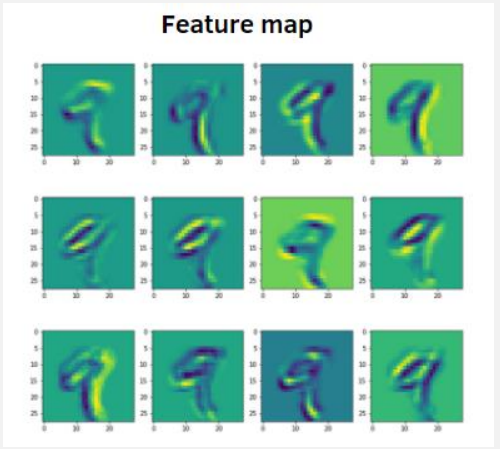
- 卷積運算會從輸入影像萃取出各種小區塊pattern，當CNN對整張影像都做完萃取後，就會產生輸出特徵映射圖(output feature map)

灰階圖像



2個空間軸(高、寬)、
1個色深度軸(channel)

RGB影像



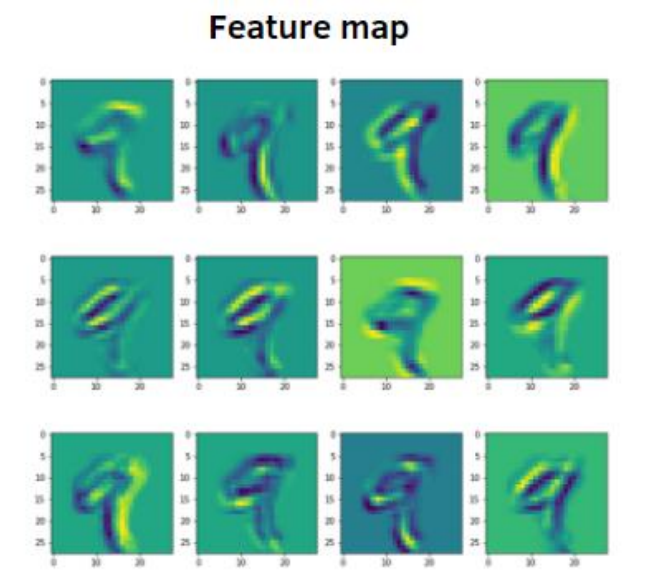
2個空間軸(高、寬)、
3個色深度軸(紅、綠、藍)

※特徵映射圖是一種抽象映射動作，是一個特徵萃取過程跟能力，不是一張圖而已

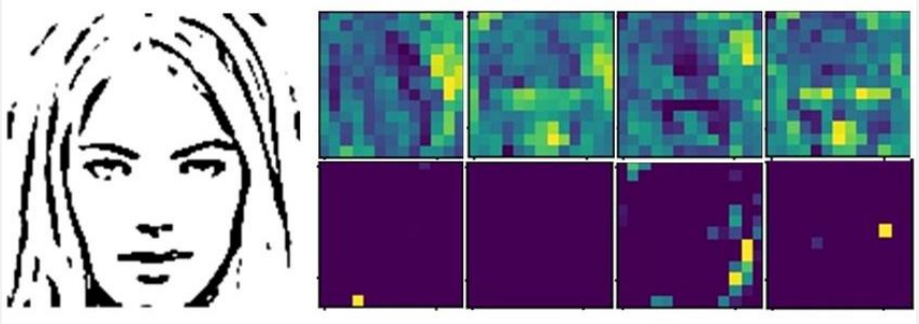
5-1-1 卷積層 - 過濾器

✓ 輸出特徵映射圖後，深度軸不再是RGB顏色值，而是代表**過濾器(filter)**

✓ 每一種過濾器會對輸入資料進行特定面向的編碼萃取出結果



MNIST範例中，
卷積層對輸入資料做了32種過濾器(filters)運算



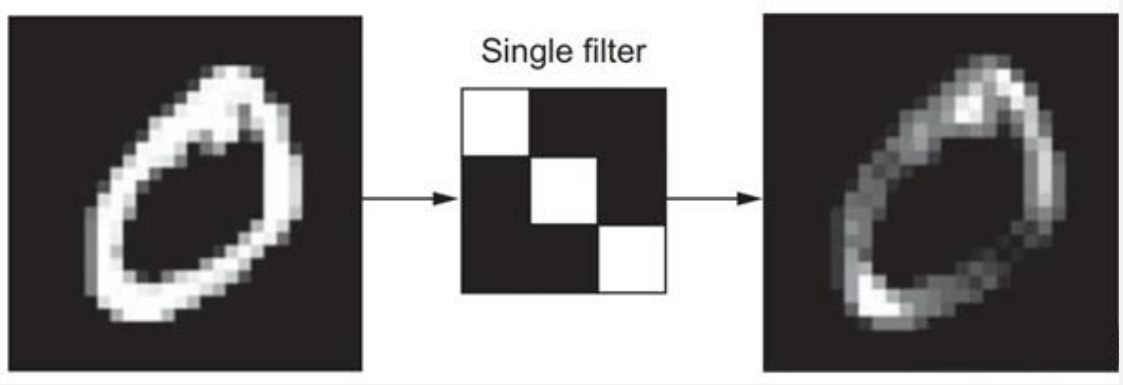
如：過濾器可以萃取到「在輸入資料中出現一張臉」這樣高階抽象的概念，而不是將臉都過濾掉

5-1-1 卷積層 - 響應圖

✓ 每個過濾器對於輸入資料的響應圖(Response map)

- 表示過濾器在輸入影像各個位置(滑動)取得的回應值(經由卷積運算)所組成的影像

原始輸入　　單一過濾器　　響應圖，表示在各個位置上要過濾的圖案是否有出現

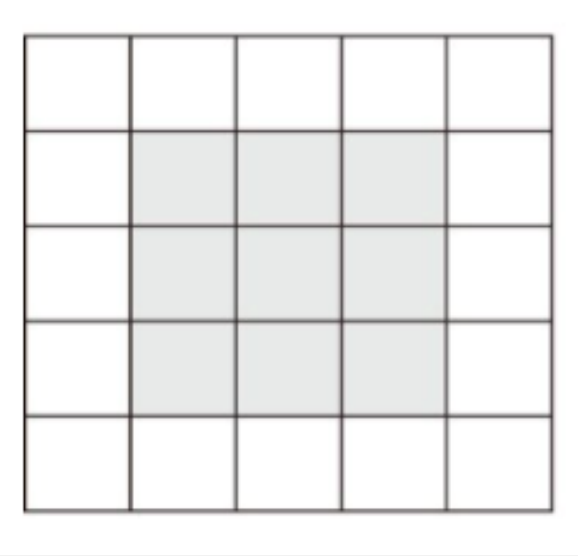


響應圖的概念：輸入影像中是否有出現filter pattern的2D響應圖

5-1-1 卷積層 – 關鍵參數

✓ 卷積層由兩個關鍵參數定義：

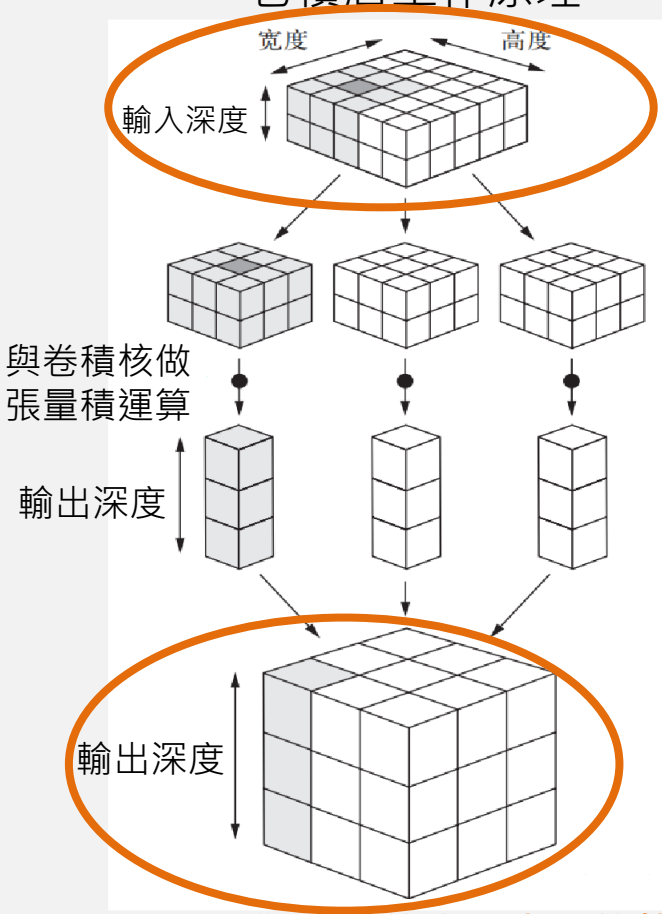
- **從輸入採樣的區塊大小**：
  - ✓ 掃描窗格的大小 (window_height, window_width)
  - ✓ 通常是3x3或5x5
  - ✓ 區塊大小(過濾器大小)為3x3是很常見的選擇
- **輸出特徵圖的深度**：
  - ✓ 卷積層的過濾器數量



5-1-1 卷積層操作 – 原理

- ✓ 卷積層主要工作步驟：
  - Step1
    - ✓ 透過在3D輸入特徵圖上滑動的小窗格，在每個可能位置停住，萃取窗格上3D區塊的特徵
  - Step2
    - ✓ 將每個3D區塊轉換做張量成shape = (output_depth)的1D向量
  - Step3
    - ✓ 將所有向量依照空間上的位置排列重新組裝成3D輸出特徵圖

卷積層工作原理



ps: 輸入(input)與輸出(output)的寬度、高度不同，可能原因為**邊界效應、步長的使用**

5-1-1 卷積層 – 邊界效應

✓ 邊界(border)效應

- 假設一個5×5 的特徵圖（共25 個方塊）且滑動窗格為3×3
- 如果窗格一次只能滑動一步，那麼就只能左右和上下各滑動3步
  - ✓ 致使輸出特徵圖的尺寸是3×3，造成輸出比輸入尺寸小了一點(下右圖所示)

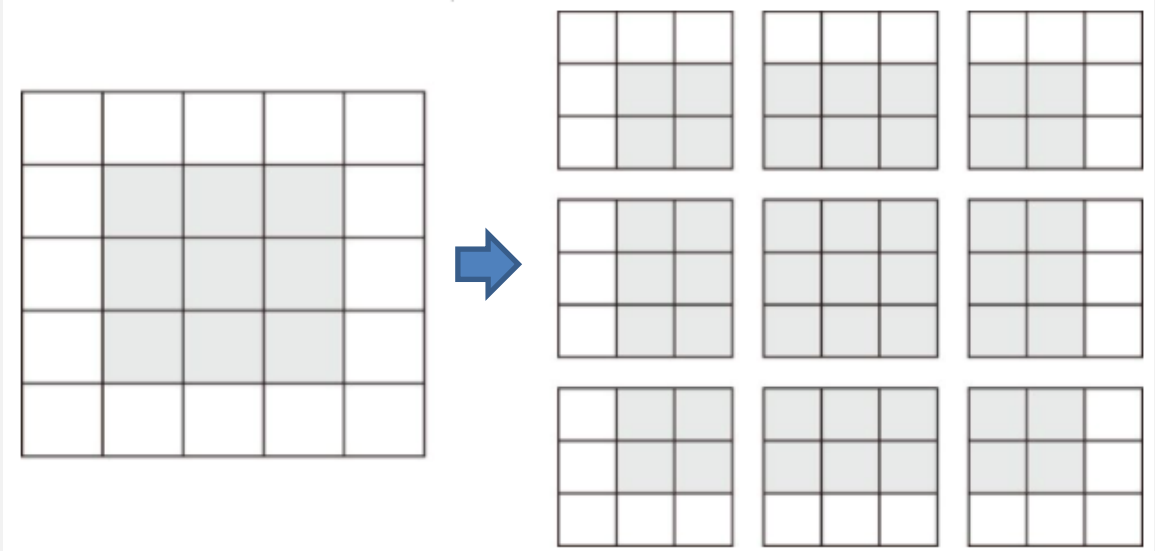


圖5–5 在5×5 的輸入特徵圖中，可以提取3×3 圖塊的有效位置

縮小兩個單位

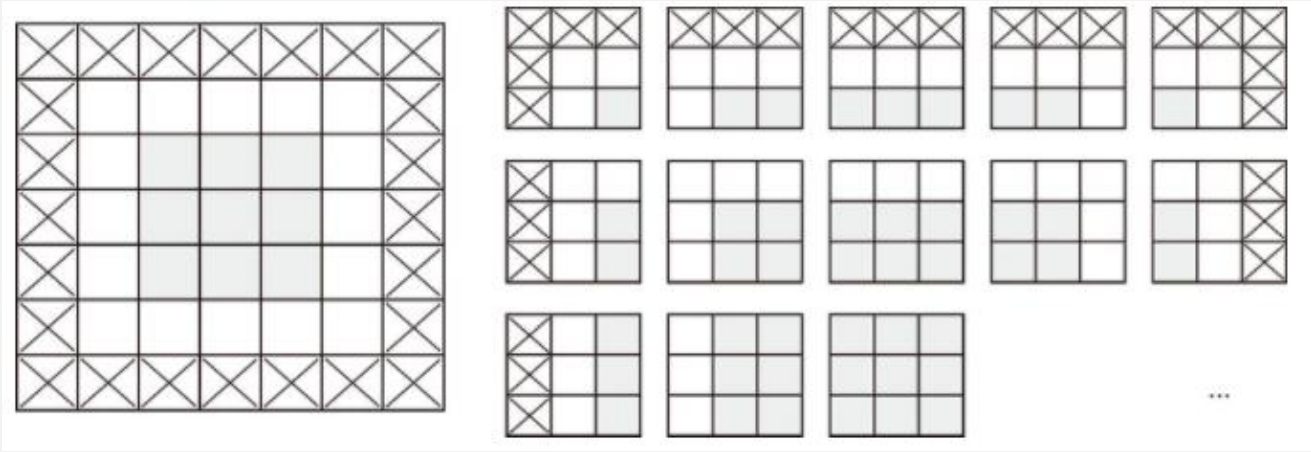
5-1-1 卷積層 – 填補

✓ 填補(padding)

- 若希望輸出特徵圖的空間維度與輸入相同，那麼可以使用**填補**（padding）
  - ✓ 在特徵圖的每一邊添加適當數目的行和列，使每個輸入方塊都能作為卷積窗口的中心

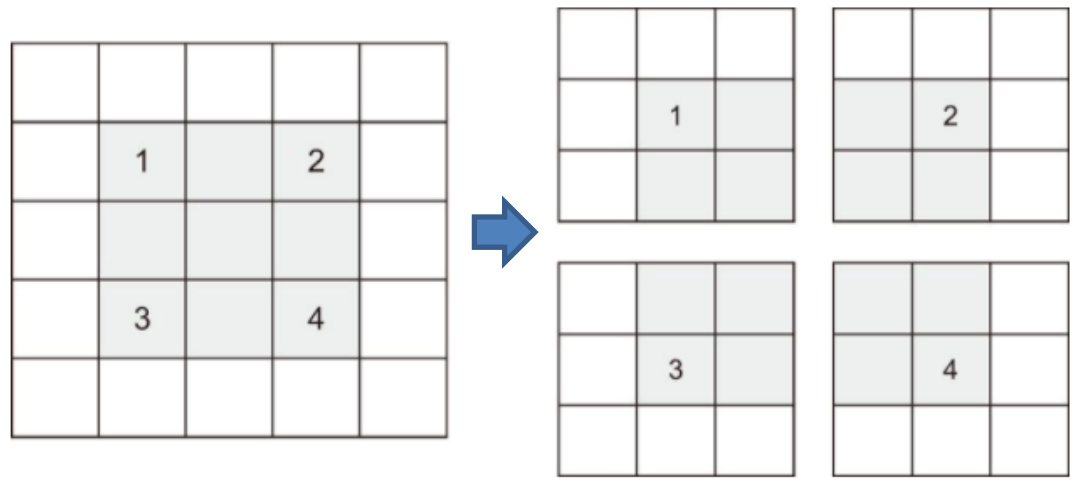
✓ 填補(padding)

- valid: 不填補(預設)
- same: 輸入同於輸出



5-1-1 卷積層 – 步長

- ✓ 步長(strides)：
  - 窗格滑動的距離，是卷積層的指定參數，預設值是1
- ✓ 步長設定卷積層(strided convolutions)：
  - 設定步長為2，可依步長參數為距離取得卷積層(即步長為2的卷積層)，如下圖：

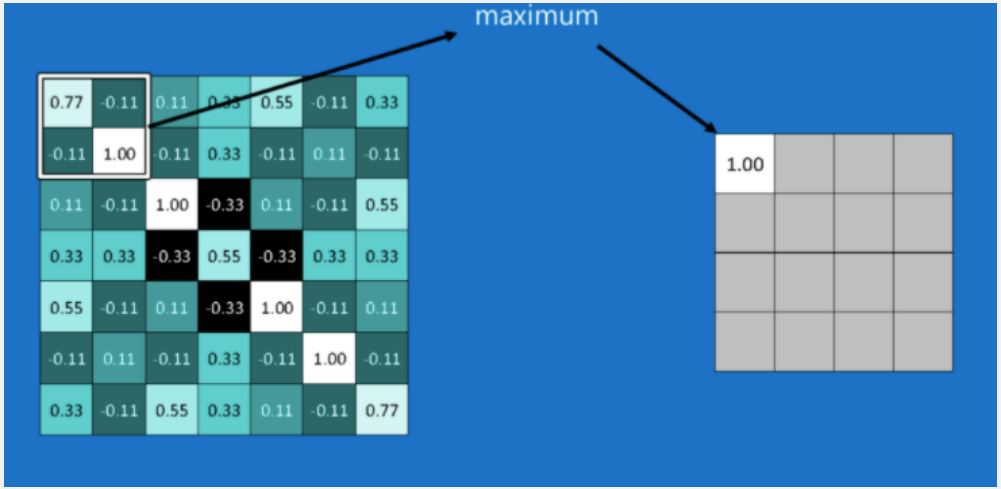


要縮小對特徵圖的採樣數，除了上述設定步長外，更常使用**最大池化(max-pooling)**操作！

5-1-2 最大化池(MaxPooling)的操作

✓ 最大池化(MaxPooling)：

- 積極地對特徵圖進行縮小採樣(採樣次數)，如同步長stride的效果
- 從輸入特徵圖中做採樣並輸出樣本的最大值，類似卷積層操作，但不是用卷積核張量積的方式來轉換局部區塊，而是經由**手動編碼的max張量**操作進行轉換
- 原圖經過池化後，特徵圖的長寬減半
- 池化後的圖片包含原圖各個範圍的最大值，故保留了各範圍與特徵的相符程度
- 池化後的資訊更專注於圖片中是否存在相符的特徵，而非哪裡存在這些特徵



5-1-2 最大化池(MaxPooling)的操作



- ✓ 不使用最大池化(MaxPooling)的問題
  - 不利於學習特徵的空間層次結構
    - ✓ 每次僅降低兩單位，無法有效學習高階的Pattern
  - 最終特徵圖扁平化後每個樣本的總係數過高, e.g., 22*22*64 = 30976
    - ✓ 模型最後傳送至Dense層，如無最大池化將造成維度過高
    - ✓ e.g. 512節點的Dense層，將需要1580萬個參數

5-2 以少量資料集從頭訓練一個卷積神經網路

- ✓ 常見於用少量資料來**訓練影像分類**的model
  - 少量樣本：幾百到幾萬張圖片的範圍

- ✓ 實作範例策略
  - 以2000樣本訓練最單純小型CNN，不使用正規化
  - 準確率為71%
  - 主要問題：overfitting

- ✓ 減少overfitting並提升神經網路準確率
  - **資料擴增法**：常用於減輕電腦視覺overfitting (提升至準確率82%)
  - **預先訓練神經網路的特徵萃取法**：用於少量資料集 (準確率90%~96%)
  - **微調預先訓練神經網路法**：用於深度學習少量資料集 (準確率97%)

5-2-1 深度學習與少量資料間的相關性

- ✓ **深度學習僅在有大量資料時才有效？**
  - Ans：一部分對 (尤其像是圖片這類高維度的輸入樣本)
- ✓ 反例：
  - CNN可學習局部pattern且具平移不變的特徵
  - ex：在少量且不必處理特徵的影像下仍可產生合理結果

- ✓ 深度學習模型可高度再利用
  - ex：大規模資料及訓練影像分類模型、語音轉文字模型



影像尺寸沒有做任何修改，因此樣本圖片的尺寸、外觀都可能不相同

5-2-2 建立相關資料夾

✓ 包含25000貓與狗的照片(各12500張)
✓ 建立訓練集(各1000)、驗證集(各500) 、測試集(各500)

```
import os, shutil
# 解壓縮資料夾所在的目錄路徑
original_dataset_dir = r'C:\Users\chche\ch05'
# 用來儲存少量資料集的目錄位置
base_dir = r'C:\Users\chche\ch05\cats_and_dogs_small'
if not os.path.isdir(base_dir): # 如果目錄不存在, 才建立目錄
    os.mkdir(base_dir)
# 分拆成訓練、驗證與測試目錄位置
train_dir = os.path.join(base_dir, 'train')   # os.path.join把檔名加上目錄的路徑，產生檔案的絕對路徑
if not os.path.isdir(train_dir):
    os.mkdir(train_dir)
validation_dir = os.path.join(base_dir, 'validation')
if not os.path.isdir(validation_dir):
    os.mkdir(validation_dir)
test_dir = os.path.join(base_dir, 'test')
if not os.path.isdir(test_dir): os.mkdir(test_dir)
```

Make directory

5-2-2 建立相關資料夾(Cont.)



- ✓ 複製圖片到訓練、驗證和測試集目錄
- ✓ 分拆成訓練、驗證與測試目錄位置

```
train_cats_dir = os.path.join(train_dir, 'cats')
if not os.path.isdir(train_cats_dir):
    os.mkdir(train_cats_dir) # 用來訓練貓圖片的目錄位置

train_dogs_dir = os.path.join(train_dir, 'dogs')
if not os.path.isdir(train_dogs_dir):
    os.mkdir(train_dogs_dir) # 用來訓練狗圖片的目錄位置

validation_cats_dir = os.path.join(validation_dir, 'cats')
if not os.path.isdir(validation_cats_dir):
    os.mkdir(validation_cats_dir) # 用來驗證貓圖片的目錄位置

validation_dogs_dir = os.path.join(validation_dir, 'dogs')
if not os.path.isdir(validation_dogs_dir):
    os.mkdir(validation_dogs_dir) # 用來驗證狗圖片的目錄位置
```

5-2-2 建立相關資料夾(Cont.)



✓ (續)分拆成訓練、驗證與測試目錄位置

```
test_cats_dir = os.path.join(test_dir, 'cats')
if not os.path.isdir(test_cats_dir):
    os.mkdir(test_cats_dir) # 用來測試貓圖片的目錄位置

test_dogs_dir = os.path.join(test_dir, 'dogs')
if not os.path.isdir(test_dogs_dir):
    os.mkdir(test_dogs_dir) # 用來測試狗圖片的目錄位置
```

5-2-2 複製資料



✓ 訓練、驗證、測試目錄位置(貓)

```
# 複製前面 1000 張貓圖片到 train_cats_dir 訓練目錄
fnames = ['cat.{}.jpg'.format(i) for i in range(1000)]
for fname in fnames:
    src = os.path.join(original_dataset_dir, fname)
    dst = os.path.join(train_cats_dir, fname)
    shutil.copyfile(src, dst) #從src複製到dst
```

5-2-2 複製資料(Cont.)



✓ (續)訓練、驗證、測試目錄位置(貓)

```
# 複製下 500 張貓圖片到 validation_cats_dir 驗證目錄
fnames = ['cat.{}.jpg'.format(i) for i in range(1000, 1500)]
for fname in fnames:
    src = os.path.join(original_dataset_dir, fname)
    dst = os.path.join(validation_cats_dir, fname)
    shutil.copyfile(src, dst)

# 複製下 500 張貓圖片到 test_cats_dir 測試目錄
fnames = ['cat.{}.jpg'.format(i) for i in range(1500, 2000)]
for fname in fnames:
    src = os.path.join(original_dataset_dir, fname)
    dst = os.path.join(test_cats_dir, fname)
    shutil.copyfile(src, dst)
```

5-2-2 複製資料(Cont.)



✓ (續)訓練、驗證、測試目錄位置(狗)

```
# 複製前面 1000 張狗圖片到 train_dogs_dir 訓練目錄
fnames = ['dog.{}.jpg'.format(i) for i in range(1000)]
for fname in fnames:
    src = os.path.join(original_dataset_dir, fname)
    dst = os.path.join(train_dogs_dir, fname)
    shutil.copyfile(src, dst)

# 複製下 500 張狗圖片到 validation_dogs_dir 驗證目錄
fnames = ['dog.{}.jpg'.format(i) for i in range(1000, 1500)]
for fname in fnames:
    src = os.path.join(original_dataset_dir, fname)
    dst = os.path.join(validation_dogs_dir, fname)
    shutil.copyfile(src, dst)
```

5-2-2 複製資料(Cont.)



✓ (續)訓練、驗證、測試目錄位置(狗)

```
# 複製下 500 張狗圖片到 test_dogs_dir 測試目錄
fnames = ['dog.{}.jpg'.format(i) for i in range(1500, 2000)]
for fname in fnames:
    src = os.path.join(original_dataset_dir, fname)
    dst = os.path.join(test_dogs_dir, fname)
    shutil.copyfile(src, dst)

print('複製完成')
```

Out[] :複製完成

5-2-2 下載資料-資料完整性檢查

```
print('訓練用的貓照片張數:', len(os.listdir(train_cats_dir)))
print('訓練用的狗照片張數:', len(os.listdir(train_dogs_dir)))
print('驗證用的貓照片張數:', len(os.listdir(validation_cats_dir)))
print('驗證用的狗照片張數:', len(os.listdir(validation_dogs_dir)))
print('測試用的貓照片張數:', len(os.listdir(test_cats_dir)))
print('測試用的狗照片張數:', len(os.listdir(test_dogs_dir)))
```

取得檔案列表

```
Out[] :
訓練用的貓照片張數: 1000
訓練用的狗照片張數: 1000
驗證用的貓照片張數: 500
驗證用的狗照片張數: 500
測試用的貓照片張數: 500
測試用的狗照片張數: 500
```

5-2-3 建立神經網路



✓ Conv2D+MaxPooling2D

- 為了處理更大影像、複雜的問題
- 增強神經網路能量、降低特徵圖的大小，避免到達展平層(Flatten)時不會過大

```
from keras import layers
from keras import models

model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
```

32個過濾器
過濾器長寬3*3

150*150像素
顏色軸的維度為3(RGB3原色)

補充：

- Conv2D把輸入資料分解成小小的2D窗格，通常以3*3、5*5
- MaxPooling2D對特徵圖進行縮小採樣，通常以2*2

5-2-3 建立神經網路(Cont.)

```
model.add(layers.Flatten())
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
model.summary() # 查看模型摘要
```

```
from keras import optimizers

model.compile(loss='binary_crossentropy',
optimizer=optimizers.RMSprop(lr=1e-4),
metrics=['acc'])
```

學習率

補充：

- 單一單元層與sigmoid激發函數結束神經網路
  - ✓ 為了解決二分類問題
  - ✓ 輸出分類的機率值
- 特徵圖深度從32到128逐漸增加，大小則從150*150到7*7逐漸變小

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_4 (Conv2D)            (None, 148, 148, 32)      896
_________________________________________________________________
max_pooling2d_4 (MaxPooling2 (None, 74, 74, 32)        0
_________________________________________________________________
conv2d_5 (Conv2D)            (None, 72, 72, 64)        18496
_________________________________________________________________
max_pooling2d_5 (MaxPooling2 (None, 36, 36, 64)        0
_________________________________________________________________
conv2d_6 (Conv2D)            (None, 34, 34, 128)       73856
_________________________________________________________________
max_pooling2d_6 (MaxPooling2 (None, 17, 17, 128)       0
_________________________________________________________________
conv2d_7 (Conv2D)            (None, 15, 15, 128)       147584
_________________________________________________________________
max_pooling2d_7 (MaxPooling2 (None, 7, 7, 128)         0
_________________________________________________________________
flatten_1 (Flatten)          (None, 6272)              0
_________________________________________________________________
dense_2 (Dense)              (None, 512)               3211776
_________________________________________________________________
dense_3 (Dense)              (None, 1)                 513
=================================================================
Total params: 3,453,121
Trainable params: 3,453,121
Non-trainable params: 0
```

5-2-4 資料預先處理



- ✓ 送入神經網路的處理步驟
  - 讀取影像檔案
  - 將JPEG內容解碼為RGB的像素
  - 將RGB像素轉換成浮點數張量
  - 將像素值(0~255)壓縮到[0,1]區間 (神經網路更適合處理小輸入值)

- ✓ Keras.preprocessing.image 模組含 ImageDataGenerate 類別
  - 自動將影像檔轉換成批次量的張量

5-2-4 資料預先處理



```
from keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(rescale=1./255) #設定訓練、測試資料的 Python 產生器，並將圖片像素值依 1/255 比例重新壓縮到 [0, 1]
test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    train_dir, # 目標目錄
    target_size=(150, 150), # 調整所有影像大小成 150x150
    batch_size=20,
    class_mode='binary') # 因為使用二元交叉熵 binary_crossentropy 作為損失值，所以需要二位元標籤

validation_generator = test_datagen.flow_from_directory(
    validation_dir,
    target_size=(150, 150),
    batch_size=20,
    class_mode='binary')
```

5-2-4 資料預先處理



✓ 產生器輸出結果

```
for data_batch, labels_batch in train_generator:
    print('data batch shape:', data_batch.shape)
    print('labels batch shape:', labels_batch.shape)
    break
```

Out[] :

```
data batch shape: (20, 150, 150, 3)
labels batch shape: (20,)
```

#每批次量20樣本，150*150像素，RGB維度3

#二元標籤的批次量

補充 :

- 產生器會無止盡的生成批次量樣本，故須在某時間點中斷(break)此循環操作

5-2-4 資料預先處理



✓ 調整模型以使用批次量產生器
- 使用fit_generator 方法，功能相當於資料產生器fit方法

```
history = model.fit_generator(
    train_generator,    #在此設定產生器
    steps_per_epoch=100,   #設定產生器抽取100個批次量 (共2000個樣本)
    epochs=30,
    validation_data=validation_generator, #允許傳入一個資料產生器
    validation_steps=50) #需設定抽取多少批次量
```

✓ 儲存模型

```
model.save('cats_and_dogs_small_1.h5')
```

5-2-4 繪製圖片

✓ 繪製訓練與驗證週期的損失值與準確度曲線

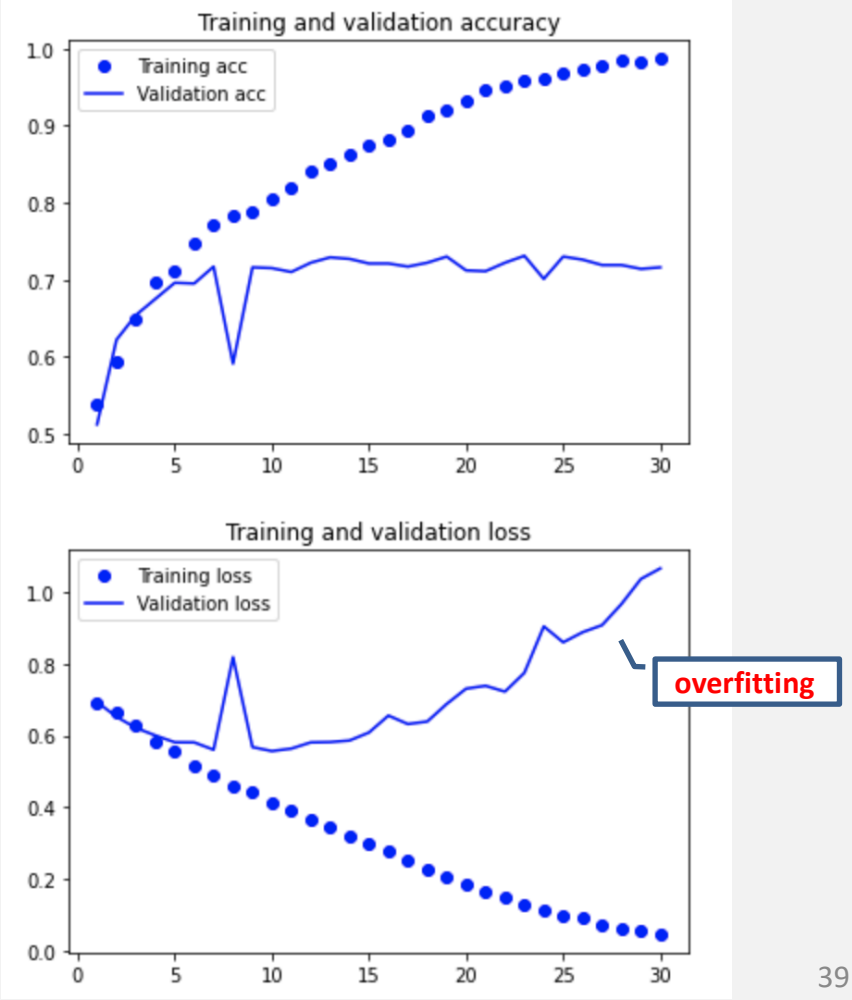
```
import matplotlib.pyplot as plt

acc = history.history['acc']
val_acc = history.history['val_acc']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.legend()
plt.figure()

plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.legend()
plt.show()
```



5-2-5 使用資料擴增法



✓ Overfitting：樣本太少無法訓練出具普適性、可套用到新資料的模型

✓ 資料擴增法(data augmentation)：從現有訓練樣本生成更多訓練資料
- 隨機變換原始資料以產生相似影像
- 最終目標：訓練時，模型永遠不會看到兩次相同的影像

在Keras中可透過 ImageDataGenerator 設定資料擴增

```
datagen = ImageDataGenerator(
    rotation_range=40,        #隨意旋轉影像角度(0~180)
    width_shift_range=0.2,    #垂直或水平隨機平移影像總寬度的20%
    height_shift_range=0.2,   #垂直或水平隨機平移影像總高度的20%
    shear_range=0.2,          #隨機順時針傾斜影像20度
    zoom_range=0.2,           #隨機縮放影像20%
    horizontal_flip=True,     #隨機水平翻轉一半影像 (非左右對稱才有效)
    fill_mode='nearest')      #新建影像填補像素(旋轉或偏移影像出現空白時使用)
```

補充 : fill_mode

costant : 依照輸入cval(浮點數或整數)填補
cval=k, kkkkkkk|abcd|kkkkkkk

nearest : 以最接近像素值填補
aaaaaaa|abcd|ddddddd

reflect : 影像重複填補(一正一反)
abcddcba|abcd|dcbaabcd

wrap : 影像重複填補(同方向)
abcdabcd|abcd|abcdabcd

5-2-5 使用資料擴增法

✓ 顯示隨機擴充的訓練影像

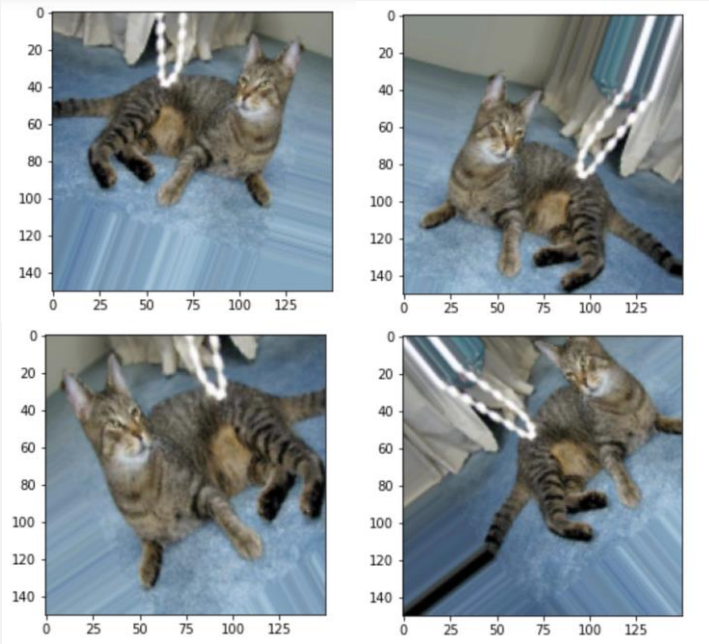
```
import keras.utils as image  #影像預先處理工具模組
from keras.utils import image_utils

fnames = [os.path.join(train_cats_dir, fname) for
    fname in os.listdir(train_cats_dir)]
img_path = fnames[3]  #選擇一張影像來擴充
img = image.load_img(img_path, target_size=(150, 150))    #讀取影像並調整大小

x = image.img_to_array(img)    #轉成shape=(150,150,3)的numpy矩陣
x = x.reshape((1, ) + x.shape)    #調整shape=(1,150,150,3) 1張150*150像素的彩色影像

i = 0
for batch in datagen.flow(x, batch_size=1):
    plt.figure(i)
    imgplot = plt.imshow(image.array_to_img(batch[0]))
    i += 1
    if i % 4 == 0:
        break
plt.show()
```

產生批次量的隨機轉換影像，預測為無限循環產生，此設定第四次停止



透過隨機資料擴增而產生的貓圖片

5-2-5 使用資料擴增法



✓ 定義具Dropout層的CNN

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu',
input_shape=(150, 150, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dropout(0.5))  #在這裡加入 Dropout 層 (丟棄 50 %)
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy',
optimizer=optimizers.RMSprop(lr=1e-4),
metrics=['acc'])
```

5-2-5 使用資料擴增法



✓ 使用資料擴增產生器來訓練CNN

```
train_datagen = ImageDataGenerator(
rescale=1./255,
rotation_range=40,
width_shift_range=0.2,
height_shift_range=0.2,
shear_range=0.2,
zoom_range=0.2,
horizontal_flip=True, )

test_datagen = ImageDataGenerator(rescale=1./255)  # 請注意！驗證資料不應該擴充!!!
```

5-2-5 使用資料擴增法

✓ (續)使用資料擴增產生器來訓練CNN

```
train_generator = train_datagen.flow_from_directory(
train_dir,   # 目標目錄
target_size=(150, 150), # 所有圖像大小調整成 150×150
batch_size=20,
class_mode='binary') # 因為使用二元交叉熵 binary_crossentropy 作為損失，所以需要二元標籤

validation_generator = test_datagen.flow_from_directory(
validation_dir,
target_size=(150, 150),
batch_size=20,
class_mode='binary')
```

5-2-5 使用資料擴增法



✓ 使用資料擴增產生器來訓練CNN

```
# 訓練
history = model.fit_generator(
train_generator,
steps_per_epoch=100,
epochs=100,
validation_data=validation_generator,
validation_steps=50)
```

✓ 儲存模型

```
model.save('cats_and_dogs_small_2.h5')
```

5-2-5 繪製圖表

✓ 繪製訓練與驗證週期的損失值與準確度曲線

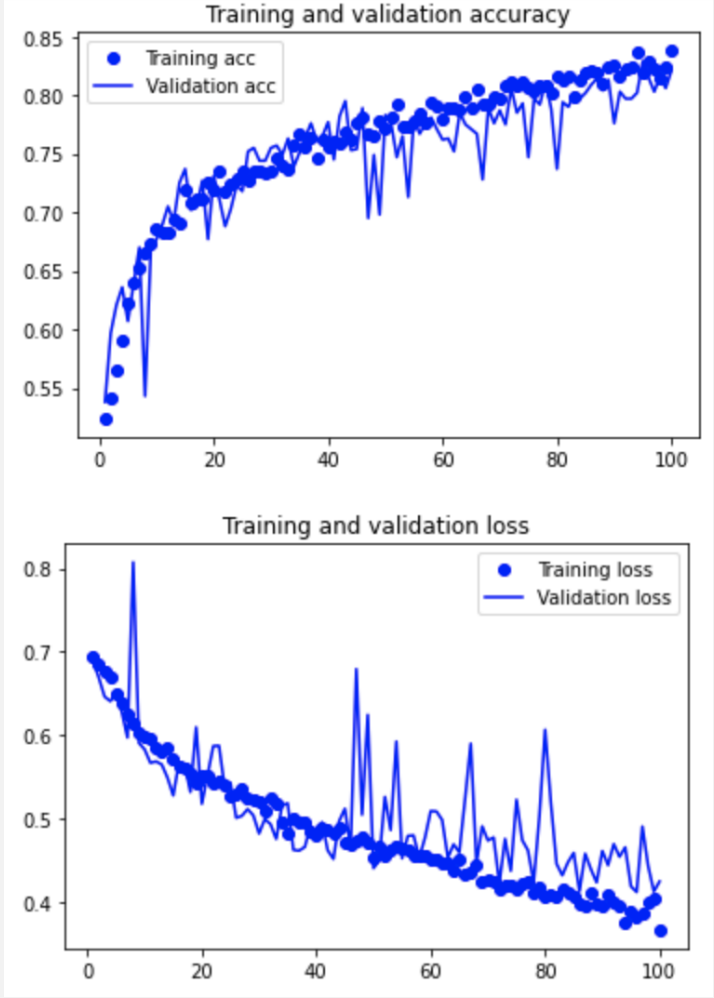
```
import matplotlib.pyplot as plt

acc = history.history['acc']
val_acc = history.history['val_acc']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.legend()
plt.figure()

plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.legend()
plt.show()
```



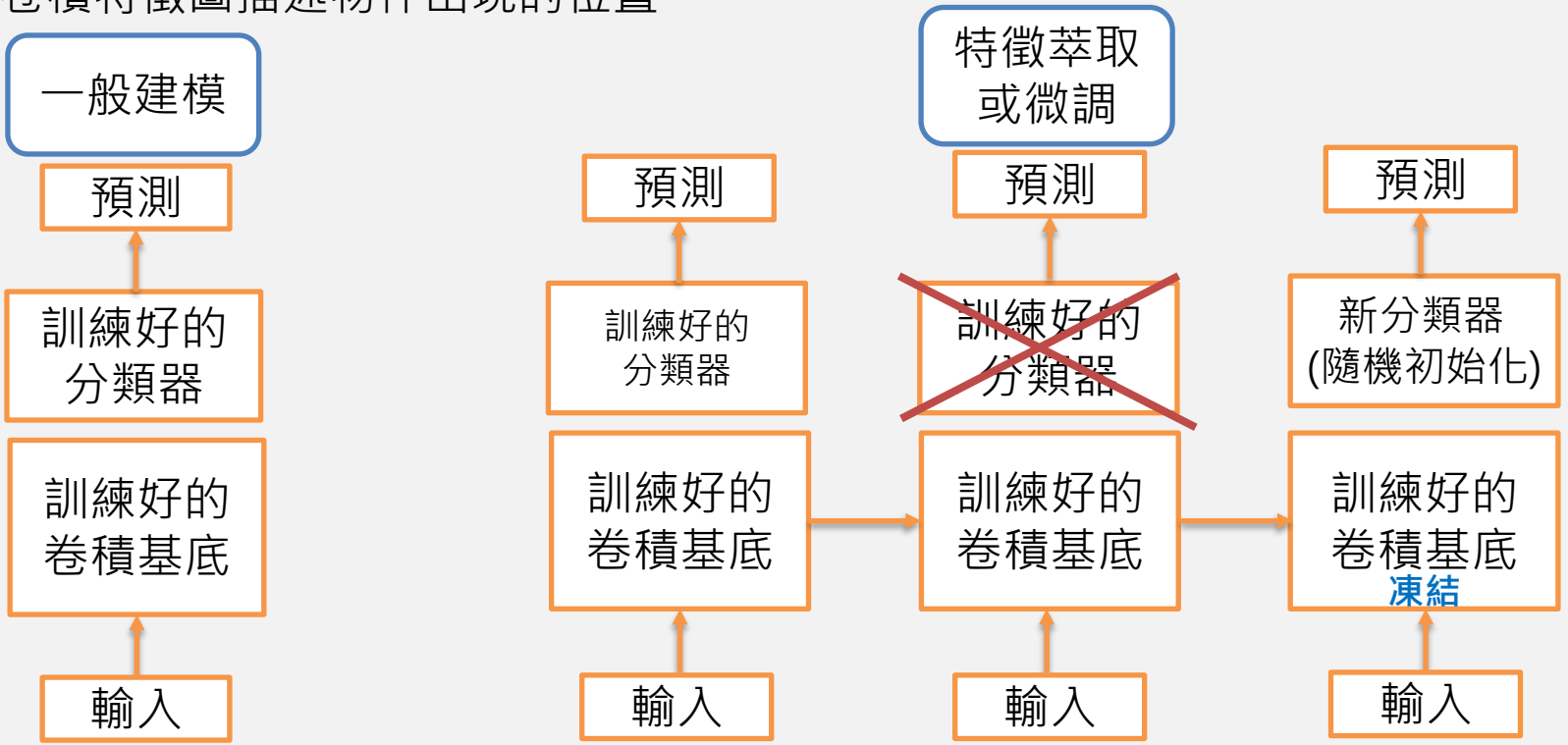
5-3 使用預先訓練的卷積神經網路

- ✓ 進行少量圖片資料集的深度學習→用預先訓練的神經網路
- ✓ 原始資料量夠大且具通用性
  - 得到空間層次特徵(spartial hierarchy features)
  - 可做為通用性的模型
  - 例子：VGG16架構
- ✓ 深度學習的優勢→學習到的特徵可以移植到不同問題上



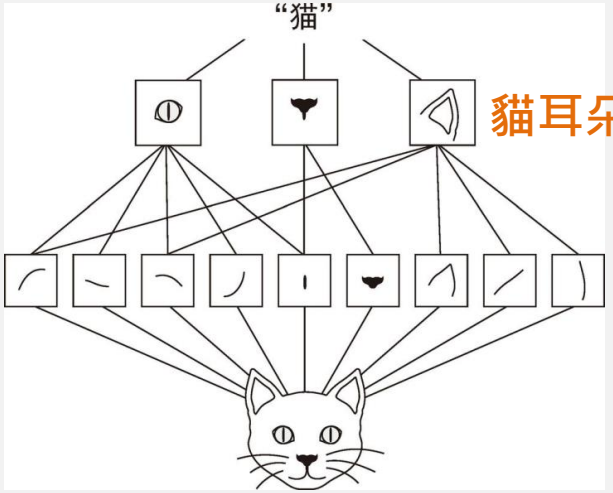
5-3-1 特徵萃取

- ✓ 透過卷積基底處理新資料，並以輸出結果訓練新的分類器
- ✓ 卷積神經網路的特徵圖→來自影像上通用模式(pattern)
- ✓ 卷積特徵圖描述物件出現的位置



5-3-1 特徵萃取

- ✓ 卷積基底學習到的表示法可能是可以通用的，因此適合重複使用
- ✓ 密集層並沒有空間概念，學習到的表示法不在包含物件在輸入影像中位置的任何訊息 ➔ 故產生的特徵絕大多數都沒用
- ✓ 特定卷積層所萃取出的表示法，其**普適性**取決於模型中層的深度：
  - ➢ 較早的層會萃取局部且高度通用的特徵圖
  - ➢ 深入的層則會萃取更抽象的概念



貓耳朵、眼睛、鼻子(抽象概念)

高度通用的特徵圖(較早層)

5-3-1 特徵萃取 – VGG16



- ✓ 實作**VGG16**卷積基底(預先安裝於Keras的模組)
- ✓ 先看看卷積基底的樣子

```
from keras.applications import VGG16

# 傳三個參數給VGG16
conv_base = VGG16(weights='imagenet',
                  include_top=False,
                  input_shape=(150, 150, 3))  #影像張量
conv_base.summary()
```

補充：

- ✓ imagenet：使用ImageNet的預先訓練的資料，約100萬張圖片，判斷1000類別的物品，如：動物、交通工具...等
- ✓ include_top：是指神經網路頂部有沒有包含密集連接層的分類器，預設是對應到ImageNet的1000個類別
- ✓ False：只利用VGG16萃取特徵，後面的分類處理都要自己設計。反之，就是全盤接受VGG16，只是要改變輸入而已

5-3-1 特徵萃取

✓ 實作VGG16卷積基底

Out[] :

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	(None, 150, 150, 3)	0
block1_conv1 (Conv2D)	(None, 150, 150, 64)	1792
block1_conv2 (Conv2D)	(None, 150, 150, 64)	36928
block1_pool (MaxPooling2D)	(None, 75, 75, 64)	0
block2_conv1 (Conv2D)	(None, 75, 75, 128)	73856
block2_conv2 (Conv2D)	(None, 75, 75, 128)	147584
block2_pool (MaxPooling2D)	(None, 37, 37, 128)	0
block3_conv1 (Conv2D)	(None, 37, 37, 256)	295168
block3_conv2 (Conv2D)	(None, 37, 37, 256)	590080
block3_conv3 (Conv2D)	(None, 37, 37, 256)	590080
block3_pool (MaxPooling2D)	(None, 18, 18, 256)	0
block4_conv1 (Conv2D)	(None, 18, 18, 512)	1180160
block4_conv2 (Conv2D)	(None, 18, 18, 512)	2359808
block4_conv3 (Conv2D)	(None, 18, 18, 512)	2359808
block4_pool (MaxPooling2D)	(None, 9, 9, 512)	0
block5_conv1 (Conv2D)	(None, 9, 9, 512)	2359808
block5_conv2 (Conv2D)	(None, 9, 9, 512)	2359808
block5_conv3 (Conv2D)	(None, 9, 9, 512)	2359808
block5_pool (MaxPooling2D)	(None, 4, 4, 512)	0

Total params: 14,714,688
Trainable params: 14,714,688
Non-trainable params: 0

補充：

✓ 最終特徵圖的shape為(4,4,512)，這是神經網路最上層的特徵，最後再接上密集連接分類器

5-3-1 沒有資料擴增的特徵萃取



- ✓ 卷積基底(convolutional base)&不使用資料擴增法
- ✓ 使用預先訓練的卷積基底萃取特徵

```
import os
import numpy as np
from keras.preprocessing.image import ImageDataGenerator

base_dir = r'D:\CH5\cats_and_dogs_small'
train_dir = os.path.join(base_dir, 'train')
validation_dir = os.path.join(base_dir, 'validation')
test_dir = os.path.join(base_dir, 'test')

#像素值（ 0和255之間）重新縮放到[0,1]間隔，進行正規化，方便deep learning做運算
datagen = ImageDataGenerator(rescale=1./255)
batch_size = 20
```

5-3-1 沒有資料擴增的特徵萃取



✓ (續)使用預先訓練的卷積基底萃取特徵

```
def extract_features(directory, sample_count): #副函式，輸入是路徑(directory)跟樣本數(sample_count)
    features = np.zeros(shape=(sample_count, 4, 4, 512)) #初始化，4*4像素的矩陣，有512個
    labels = np.zeros(shape=(sample_count))
    generator = datagen.flow_from_directory(directory, #把圖檔從路徑(directory)讀進來
                                            target_size=(150, 150),
                                            batch_size=batch_size,
                                            class_mode='binary’)
    i = 0
    for inputs_batch, labels_batch in generator: #萃取
        features_batch = conv_base.predict(inputs_batch)
        features[i * batch_size : (i + 1) * batch_size] = features_batch
        labels[i * batch_size : (i + 1) * batch_size] = labels_batch
        i += 1
        print(i, end=' ') # 由於萃取需要較長的時間，我們印出 i 來檢視進度
        if i * batch_size >= sample_count: # 當萃取樣本數超過設定的樣本數要中斷處理
            break
    return features, labels
```

5-3-1 沒有資料擴增的特徵萃取



- ✓ (續)使用預先訓練的卷積基底萃取特徵

樣本數

```
train_features, train_labels = extract_features(train_dir, 2000)
validation_features, validation_labels = extract_features(validation_dir, 1000)
test_features, test_labels = extract_features(test_dir, 1000)
```

- ✓ 目前萃取的特徵shape=(樣本數,4,4,512)，為了要提供給密集層分類器，必須將資料展平

5-3-1 沒有資料擴增的特徵萃取



✓ 將資料展平

樣本數

```
train_features = np.reshape(train_features, (2000, 4 * 4 * 512))
validation_features = np.reshape(validation_features, (1000, 4 * 4 * 512))
test_features = np.reshape(test_features, (1000, 4 * 4 * 512))
```

說明：
萃取的特徵為 shape=(樣本數, 4, 4, 512)
展平後為(樣本數, 8192)

4*4*512

5-3-1 沒有資料擴增的特徵萃取

✓ 建立和訓練密集連接的分類器

```
from keras import models
from keras import layers
from keras import optimizers

model = models.Sequential()
model.add(layers.Dense(256, activation='relu', input_dim=4 * 4 * 512))
model.add(layers.Dropout(0.5)) # 丟棄法
model.add(layers.Dense(1, activation='sigmoid'))
#重新編譯
model.compile(optimizer=optimizers.RMSprop(lr=2e-5), #優化器用RMSprop，微調所以學習率設很低
              loss='binary_crossentropy',
              metrics=['acc'])

history = model.fit(train_features, # 用卷積基底萃取的特徵資料進行訓練與驗證
                    train_labels,epochs=30,
                    batch_size=20,
                    validation_data=(validation_features, validation_labels))
```

5-3-1 沒有資料擴增的特徵萃取

✓ 繪製結果

```
import matplotlib.pyplot as plt

acc = history.history['acc']
val_acc = history.history['val_acc']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.legend()

plt.figure()
```

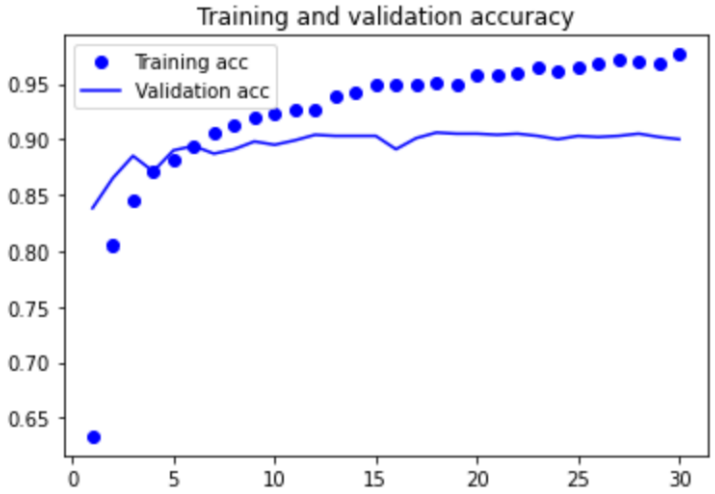
5-3-1 沒有資料擴增的特徵萃取

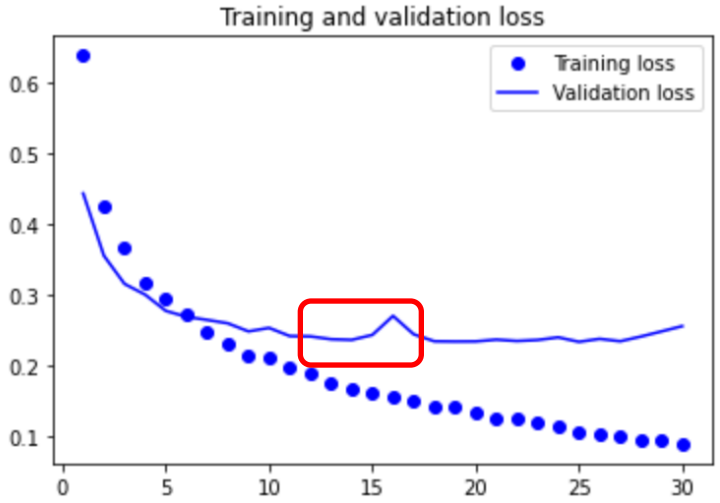
✓ (續)繪製結果

```
plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.legend()

plt.show()
```

Out[]：達90%的準確率，但是有過度配適的問題





5-3-1 資料擴增的特徵萃取



✓ 在卷積基底上增加密集層分類器 (建議在GPU上運作)

```
from keras import models
from keras import layers
from keras.applications import VGG16

conv_base = VGG16(weights='imagenet', #卷積基底
                  include_top=False,
                  input_shape=(150, 150, 3))

model = models.Sequential()
model.add(conv_base)      # 將卷積基底視為層加入 Sequential 模型中
model.add(layers.Flatten()) # 攤平
model.add(layers.Dense(256, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid')) # 增加密集層分類器
model.summary() # 查看模型摘要
```

5-3-1 資料擴增的特徵萃取

✓ (續)在卷積基底上增加密集層分類器

Out[] :

```
Model: "sequential_4"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
vgg16 (Functional)           (None, 4, 4, 512)         14714688
_________________________________________________________________
flatten_3 (Flatten)          (None, 8192)              0
_________________________________________________________________
dense_8 (Dense)              (None, 256)               2097408
_________________________________________________________________
dense_9 (Dense)              (None, 1)                 257
=================================================================
Total params: 16,812,353
Trainable params: 16,812,353
Non-trainable params: 0
```

卷積基底

增加的分類器

5-3-1 資料擴增的特徵萃取

- ✓ 凍結卷積基底神經網路
- ✓ 凍結：在訓練期間禁止更新權重
- ✓ 避免卷積基底先前學習到的表示法被修改掉
  - 頂部的密集層隨機初始化
  - 權重更新會隨著神經網路傳播

```
print('This is the number of trainable weights '
'before freezing the conv base:', len(model.trainable_weights))

conv_base.trainable = False  #凍結權重

print('This is the number of trainable weights '
'after freezing the conv base:', len(model.trainable_weights))
```

```
This is the number of trainable weights before freezing the conv base: 30
This is the number of trainable weights after freezing the conv base: 4
```

只會訓練增加的兩個Dense層的權重
總共有4個權重張量：每層2個(主要權重矩陣和偏差向量)

5-3-1 資料擴增的特徵萃取



✓ 以凍結的卷積基底進行完整的訓練

```
from keras.preprocessing.image import ImageDataGenerator
from keras import optimizers

train_datagen = ImageDataGenerator( # 擴充訓練資料
                rescale=1./255,
                rotation_range=40,
                width_shift_range=0.2,
                height_shift_range=0.2,
                shear_range=0.2,
                zoom_range=0.2,
                horizontal_flip=True,
                fill_mode='nearest')

test_datagen = ImageDataGenerator(rescale=1./255) # 請注意驗證資料不應該擴充
```

5-3-1 資料擴增的特徵萃取

✓ 以凍結的卷積基底進行完整的訓練

```
train_generator = train_datagen.flow_from_directory(
                train_dir, # 目標目錄路徑
                target_size=(150, 150), # 調整所有圖像大小成 150×150
                batch_size=20,
                class_mode='binary’)
# 因為使用二元交叉熵 binary_crossentropy 作為損失分數，所以需要二元標籤

validation_generator = test_datagen.flow_from_directory(
                validation_dir,
                target_size=(150, 150),
                batch_size=20,
                class_mode='binary')

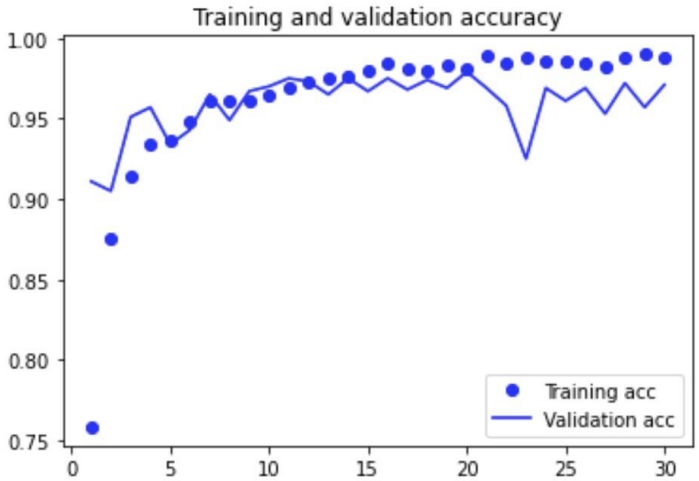
model.compile( loss='binary_crossentropy',
                                        optimizer=optimizers.RMSprop(lr=2e-5),
                                        metrics=['acc'])
```

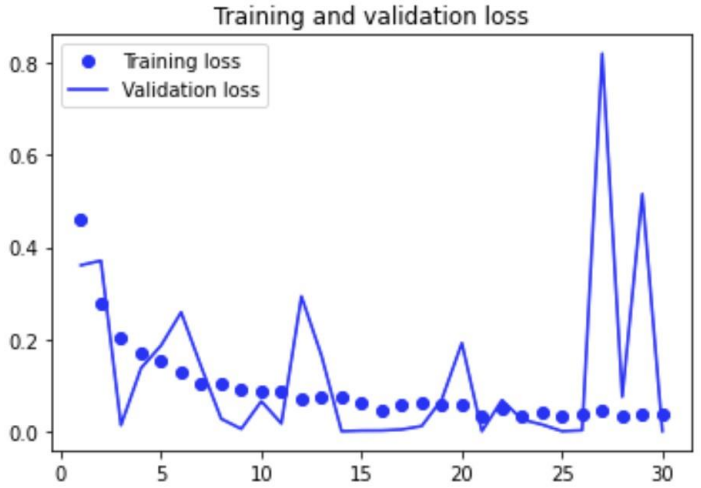
5-3-1 資料擴增的特徵萃取

✓ 以凍結的卷積基底進行完整的訓練

```
history = model.fit_generator( #訓練模型
            train_generator,
            steps_per_epoch=100,
            epochs=30,
            validation_data=validation_generator,
            validation_steps=50)
```

Out[]：達96%的準確率，比從頭訓練的小型神經網路基準能力要好很多

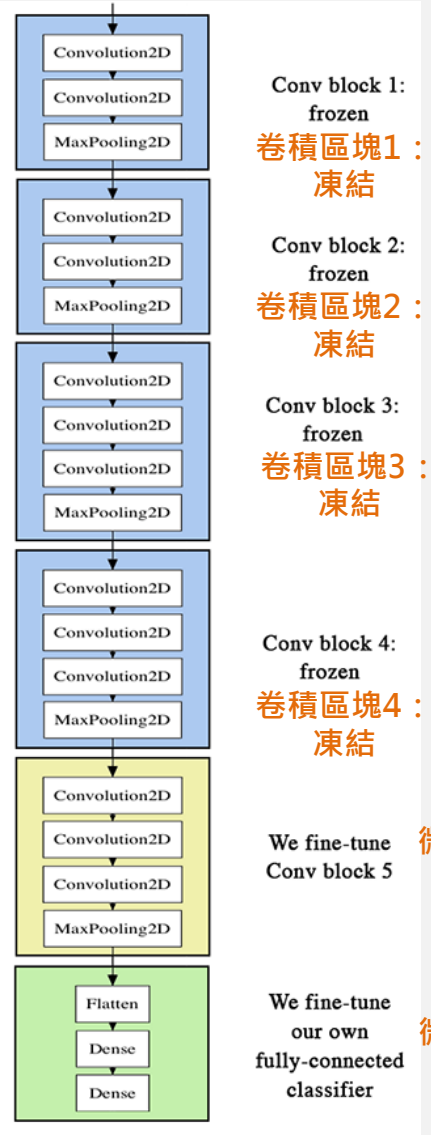




5-3-2 微調

- ✓ 另一種廣被使用的model reuse技術：
- ✓ 微調(fine-tuning)
  - 解凍**一些卷積基底頂部的某些層**做特徵萃取卷積層頂部或密集層
  - 並透過微調了原卷積基底的表示法，以便應用到欲解決的問題

微調VGG16神經網路的最後一個卷積區塊



5-3-2 微調

✓微調步驟：

- 1. 在已經訓練過的卷積基底上增加自定義神經網路
- 2. 凍結卷積基底
- 3. 訓練自定義神經網路(最頂部的分類器)
- 4. 解凍卷積基底的某幾層
- 5. 共同訓練解凍的層與最頂部的分類器

5-3-2 微調



✓ 微調最後三個卷積層

```
from keras.applications import VGG16

conv_base = VGG16(weights='imagenet', include_top=False, input_shape=(150, 150, 3))
conv_base.summary()

conv_base.trainable = True # 權重解凍
set_trainable = False
# 將所有層凍結到指定層為止
for layer in conv_base.layers:
    if layer.name == 'block5_conv1':
        set_trainable = True
    if set_trainable:
        layer.trainable = True
    else:
        layer.trainable = False
```

5-3-2 微調

Out[] :

```
Model: "vgg16"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_3 (InputLayer)         [(None, 150, 150, 3)]     0
_________________________________________________________________
block1_conv1 (Conv2D)        (None, 150, 150, 64)      1792
_________________________________________________________________
block1_conv2 (Conv2D)        (None, 150, 150, 64)      36928
_________________________________________________________________
block1_pool (MaxPooling2D)   (None, 75, 75, 64)        0
_________________________________________________________________
block2_conv1 (Conv2D)        (None, 75, 75, 128)       73856
_________________________________________________________________
block2_conv2 (Conv2D)        (None, 75, 75, 128)       147584
_________________________________________________________________
block2_pool (MaxPooling2D)   (None, 37, 37, 128)       0
_________________________________________________________________
block3_conv1 (Conv2D)        (None, 37, 37, 256)       295168
_________________________________________________________________
block3_conv2 (Conv2D)        (None, 37, 37, 256)       590080
_________________________________________________________________
block3_conv3 (Conv2D)        (None, 37, 37, 256)       590080
_________________________________________________________________
block3_pool (MaxPooling2D)   (None, 18, 18, 256)       0
_________________________________________________________________
block4_conv1 (Conv2D)        (None, 18, 18, 512)       1180160
_________________________________________________________________
block4_conv2 (Conv2D)        (None, 18, 18, 512)       2359808
_________________________________________________________________
block4_conv3 (Conv2D)        (None, 18, 18, 512)       2359808
_________________________________________________________________
block4_pool (MaxPooling2D)   (None, 9, 9, 512)         0
_________________________________________________________________
block5_conv1 (Conv2D)        (None, 9, 9, 512)         2359808
_________________________________________________________________
block5_conv2 (Conv2D)        (None, 9, 9, 512)         2359808
_________________________________________________________________
block5_conv3 (Conv2D)        (None, 9, 9, 512)         2359808
_________________________________________________________________
block5_pool (MaxPooling2D)   (None, 4, 4, 512)         0
=================================================================
Total params: 14,714,688
Trainable params: 14,714,688
Non-trainable params: 0
```

5-3-2 微調



- ✓ 微調神經網路
- ✓ 使用RMSProp優化器，並以低學習率進行微調

```
# 編譯模型
model.compile(
    loss='binary_crossentropy',
    optimizer=optimizers.RMSprop(lr=1e-5),
    metrics=['acc'])

# 訓練模型
history = model.fit_generator(
    train_generator,
    steps_per_epoch=100,
    epochs=100,
    validation_data=validation_generator,
    validation_steps=50)
```

5-3-2 微調

✓ 繪製結果

```
import matplotlib.pyplot as plt

acc = history.history['acc']
val_acc = history.history['val_acc']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.legend()

plt.figure()
```

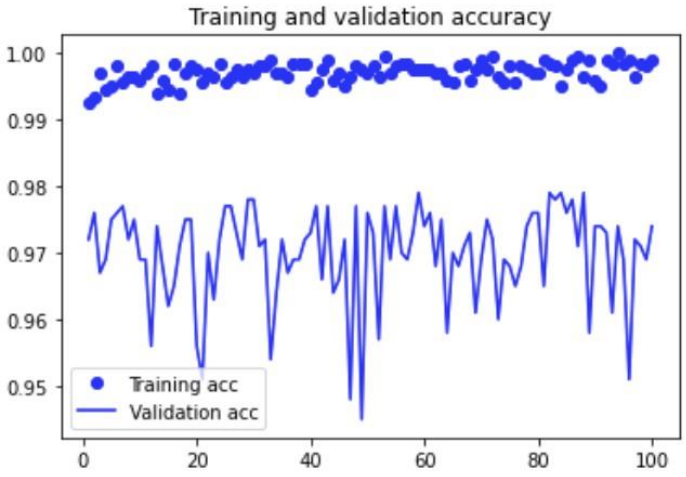
5-3-2 微調

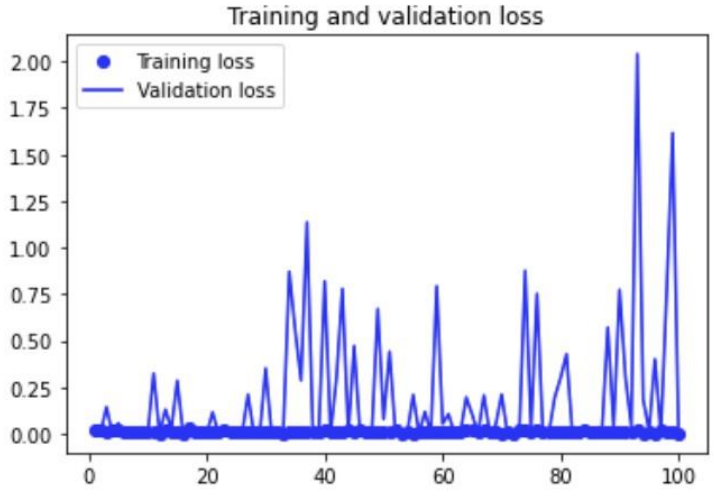
✓ (續)繪製結果

```
plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.legend()

plt.show()
```

Out[] : 這些曲線看起來很紛亂，可透過指數移動平均值增加可讀性





5-3-2 微調

✓ 利用指數移動平均，繪製平滑曲線

```
def smooth_curve(points, factor=0.8):
        smoothed_points = []
        for point in points:
                if smoothed_points:
                        previous = smoothed_points[-1]
                        smoothed_points.append(previous * factor + point * (1 - factor))
                else:
                        smoothed_points.append(point)
        return smoothed_points
```

5-3-2 微調

✓ 利用指數移動平均，繪製平滑曲線

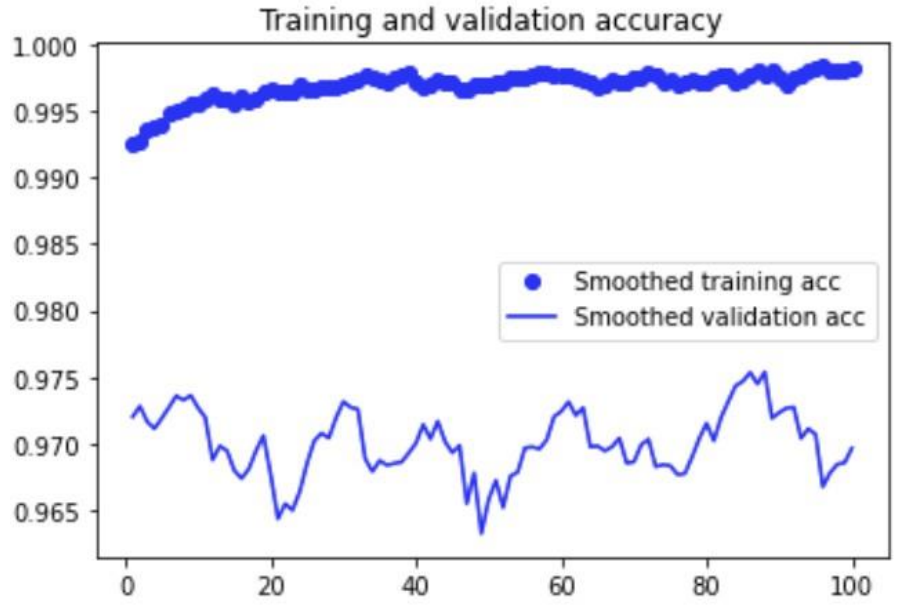
```
plt.plot( epochs, smooth_curve(acc), 'bo', label='Smoothed training acc')
plt.plot( epochs, smooth_curve(val_acc), 'b', label='Smoothed validation acc')
plt.title('Training and validation accuracy')
plt.legend()
plt.figure()

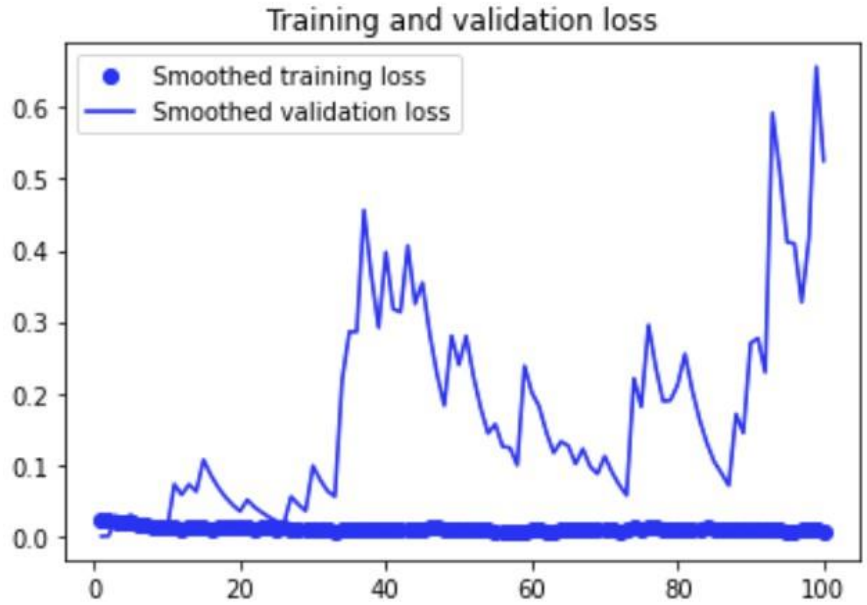
plt.plot( epochs,  smooth_curve(loss), 'bo', label='Smoothed training loss')
plt.plot( epochs,  smooth_curve(val_loss), 'b', label='Smoothed validation loss')
plt.title('Training and validation loss')
plt.legend()
plt.show()
```

5-3-2 微調

✓ 利用指數移動平均，繪製平滑曲線

Out[] :





損失沒有減少，但是準確度卻很平穩?
對準確度而言，要看的是損失值的分布，不是看損失值的平均

5-3-2 微調



✓ 使用測試資料來評估微調後的模型

```
test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(150, 150),
    batch_size=20,
    class_mode='binary')

test_loss, test_acc = model.evaluate_generator(test_generator, steps=50)
print('test acc:', test_acc)
```

Out[] :

test acc: 0.9639999866485596

說明：

可以獲得97%的測試準確度，這已是此資料集在原始Kaggle比賽中的最佳結果之一

5-4 視覺化呈現卷積神經網路學習的內容



✓ 深度學習所學到的表示法難以萃取

- 卷積神經網路(CNN)例外 – 大部分是視覺概念的表示法

✓ 三種最易取得能視覺化解釋表示法的技術

- **視覺化中間層convnet的輸出(中間啟動函數的輸出)**
  - ✓ 有助於理解convnet是如何一層一層轉化資料、對過濾器的含義有一個概念
- **視覺化卷積神經網路過濾器**
  - ✓ 用於準確理解卷積神經網路中，每個過濾器所要接受的視覺pattern或概念
- **視覺化類別激活熱圖(heatmaps of class activation)**
  - ✓ 有助於理解影像的哪些部分被辨識為某個類別，能定位影像中的物件

5-4-1 中間層輸出的視覺化

✓ 顯示出convnet各個卷積層、池化層輸出的特徵圖
- 層的輸出指activation function的輸出
- 了解如何逐層分解至不同的過濾器

✓ 顯示特徵圖的三個維度：寬度、高度、深度(channel)
- 將每個channel的內容獨立繪製成2D圖形
- 譬應圖：想像成吐司條切出一片一片的超薄吐司



5-4-1 中間層輸出的視覺化

✓ 載入5-2已建立並儲存的model

```
from keras.models import load_model
model = load_model(r'cats_and_dogs_small_2.h5')
model.summary()
```

Out[] :

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_4 (Conv2D)            (None, 148, 148, 32)      896
_________________________________________________________________
max_pooling2d_4 (MaxPooling2 (None, 74, 74, 32)        0
_________________________________________________________________
conv2d_5 (Conv2D)            (None, 72, 72, 64)        18496
_________________________________________________________________
max_pooling2d_5 (MaxPooling2 (None, 36, 36, 64)        0
_________________________________________________________________
conv2d_6 (Conv2D)            (None, 34, 34, 128)       73856
_________________________________________________________________
max_pooling2d_6 (MaxPooling2 (None, 17, 17, 128)       0
_________________________________________________________________
conv2d_7 (Conv2D)            (None, 15, 15, 128)       147584
_________________________________________________________________
max_pooling2d_7 (MaxPooling2 (None, 7, 7, 128)         0
_________________________________________________________________
flatten_1 (Flatten)          (None, 6272)              0
_________________________________________________________________
dropout (Dropout)            (None, 6272)              0
_________________________________________________________________
dense_2 (Dense)              (None, 512)               3211776
_________________________________________________________________
dense_3 (Dense)              (None, 1)                 513
=================================================================
Total params: 3,453,121
Trainable params: 3,453,121
Non-trainable params: 0
```

使用資料擴增法的模型

補充：

- Conv2D把輸入資料分解成小小的2D窗格，通常以3*3、5*5
- MaxPooling2D對特徵圖進行縮小採樣，通常以2*2

5-4-1 中間層輸出的視覺化



✓ 預處理單張照片

```
from tensorflow.keras.utils import load_img
from tensorflow.keras.utils import img_to_array
import numpy as np

img_path = r'C:\Users\chche\ch05\cats_and_dogs_small\test\cats\cat.1700.jpg'
# 將這張圖片預處理成 4D 張量並將像素值限制在 0-1 之間
img = load_img(img_path, target_size=(150, 150))  #讀取影像並調整大小
img_tensor = img_to_array(img)  #轉換成shape=(150,150,3)的numpy矩陣
img_tensor = np.expand_dims(img_tensor, axis=0)
img_tensor /= 255.  #壓縮像素值在 0-1 之間

print(img_tensor.shape)  # shape = (1, 150, 150, 3)
```

Out[] :
(1, 150, 150, 3)

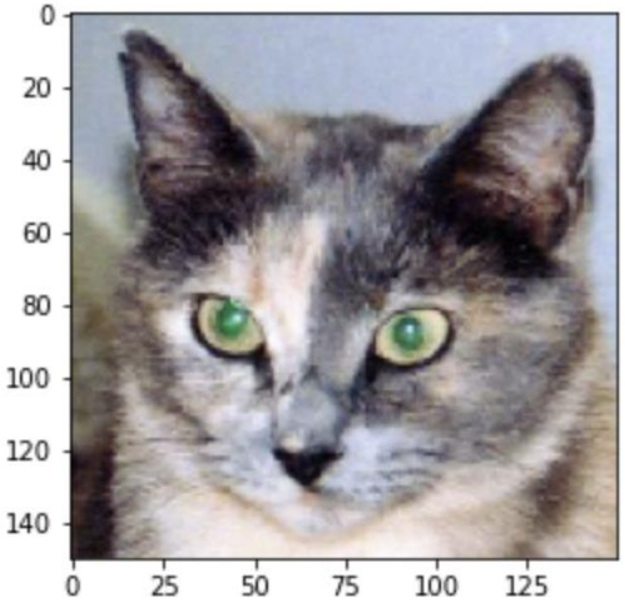
5-4-1 中間層輸出的視覺化

✓ 顯示測試影像

```
import matplotlib.pyplot as plt

plt.imshow(img_tensor[0])
plt.show()
```

Out[] :



5-4-1 中間層輸出的視覺化



- ✓ 為萃取想觀看的特徵圖，建立一個keras model
  - 以批次量的影像作為輸入，將所有卷積層、池化層的啟動函數值輸出
- ✓ 與Sequential model一樣，會將指定輸入對應到指定的輸出
  - 差異：model所建立的模型可有多個輸出
- ✓ 用一個輸入張量和一個輸出張量 list (8個層) 來建構一個model

```
from keras import models

layer_outputs = [layer.output for layer in model.layers[:8]] #萃取model前八層的輸出張量
for op in layer_outputs:
    print(op)

activation_model = models.Model(inputs=model.input, outputs=layer_outputs)
```

5-4-1 中間層輸出的視覺化



✓在predict model下執行model

```
activations = activation_model.predict(img_tensor)
#傳回一個含8個輸出張量的list，他們是8個layer的啟動函數輸出值(張量)
print(len(activations))

first_layer_activation = activations[0]  #第一個卷積層(index為0)的啟動函數輸出張量
print(first_layer_activation.shape)
```

Out[] :

8

(1, 148, 148, 32) #1個148*148的特徵圖，有32個channel

補充 :

- model是可以有任意數量的輸入與輸出
- Ex: 本例為1個輸入，8個輸出（前8層的啟動函數產生了8個輸出）

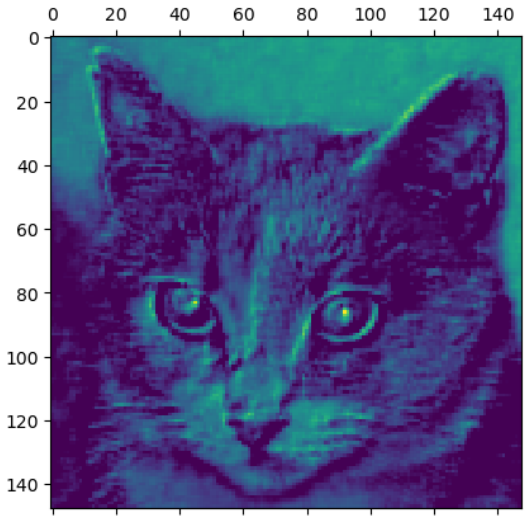
5-4-1 中間層輸出的視覺化

- ✓ 視覺化（繪製）第4個channel的響應圖
- ✓ 似乎編碼成一個邊緣特徵偵測器

```
import matplotlib.pyplot as plt

plt.matshow(first_layer_activation[0, :, :, 4], cmap='viridis')
```

Out[] :



顏色對應：紫-綠-黃

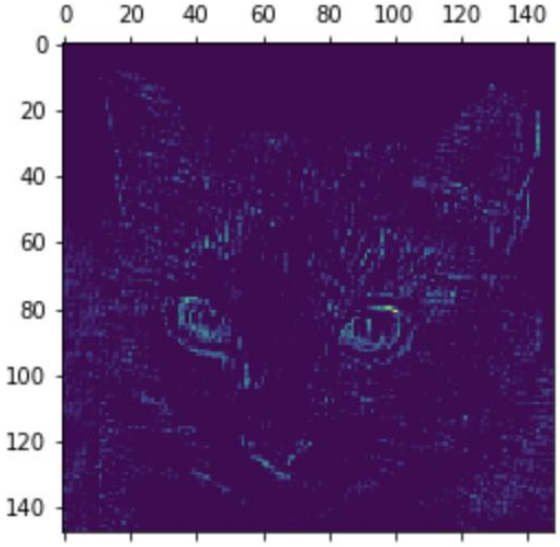
補充 :matplotlib.matshow()方法可以透過指名參數cmap來決定張量數值對應的顏色範圍

5-4-1 中間層輸出的視覺化

- ✓ 視覺化（繪製）第7個channel的響應圖
- ✓ 編碼成一個亮綠點偵測器

```
plt.matshow(first_layer_activation[0, :, :, 7], cmap='viridis')
```

Out[] :



補充：

- 可將每個channel代表某種特徵萃取結果
- 輸出張量中的channel數量代表這個層所萃取出的特徵數量（過濾器→過濾出某種特徵）

5-4-1 中間層輸出的視覺化

✓ 視覺化(繪製)每個啟動函數輸出中的每個channel的響應圖

```
layer_names = [] # 取得各層的名字，這樣才可以成為圖表的一部分

for layer in model.layers[:8]: #取得模型前8層
    layer_names.append(layer.name) #取得前8層的名字

images_per_row = 16 #在大圖中，每個橫列可以放16張圖

for layer_name, layer_activation in zip(layer_names, activations):  #顯示響應圖

   n_features = layer_activation.shape[-1]
#輸出張量的shape=(1,長,寬,channels)，channel即代表特徵數量及某層輸出張量中的特徵數量
   size = layer_activation.shape[1]   #取得響應圖的尺寸大小(正方形、長寬相同)

   n_cols = n_features // images_per_row
   display_grid = np.zeros((size * n_cols, images_per_row * size))
```

5-4-1 中間層輸出的視覺化



✓ (續)視覺化(繪製)每個啟動函數輸出中的每個channel的響應圖

```
for col in range(n_cols):    #將每個響應圖拼成一個大的水平網格
    for row in range(images_per_row):
        channel_image = layer_activation[0, :, :, col * images_per_row + row]
        channel_image -= channel_image.mean()
        channel_image /= channel_image.std()
        channel_image *= 64
        channel_image += 128
        channel_image = np.clip(channel_image, 0, 255).astype('uint8')
        display_grid[col * size : (col + 1) * size, #以網格顯示
                     row * size : (row + 1) * size] = channel_image

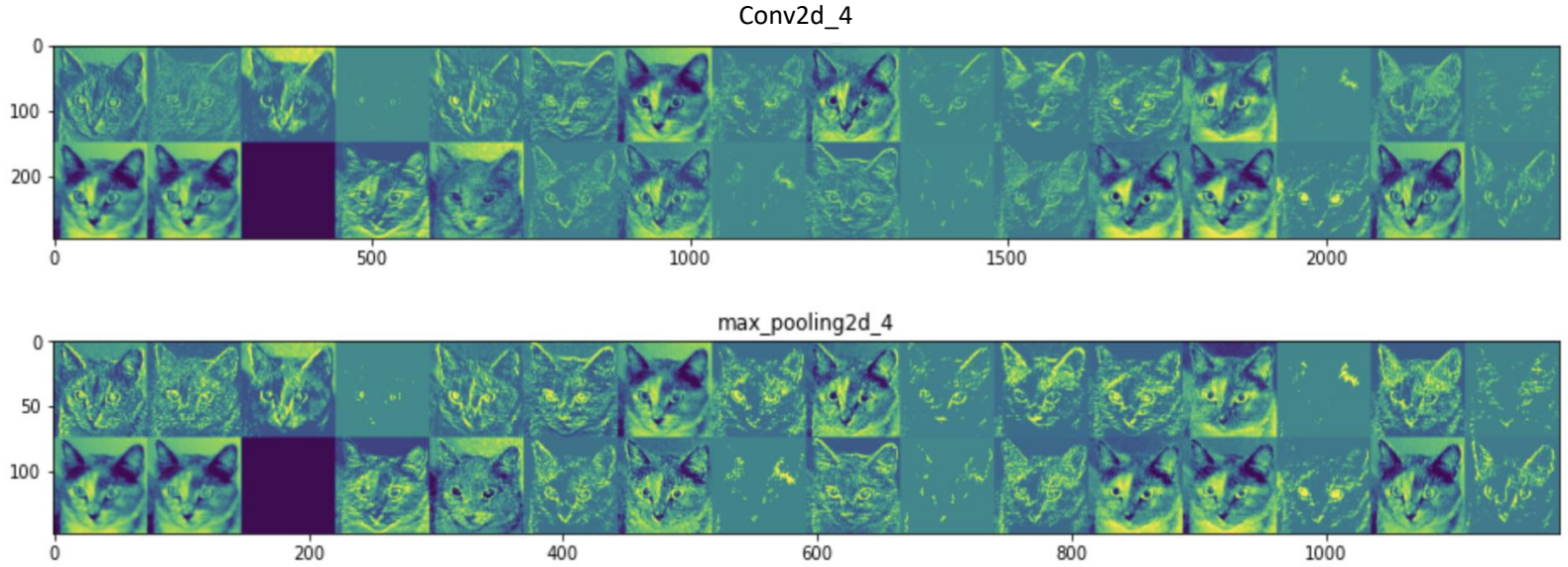
scale = 1. / size
plt.figure(figsize=(scale * display_grid.shape[1],
scale * display_grid.shape[0]))
plt.title(layer_name)
plt.grid(False)
plt.imshow(display_grid, aspect='auto', cmap='viridis')
```

對特徵進行後處理，使其在視覺上更美觀

5-4-1 中間層輸出的視覺化

✓ (續)視覺化(繪製)每個啟動函數輸出中的每個channel的響應圖

Out[] :

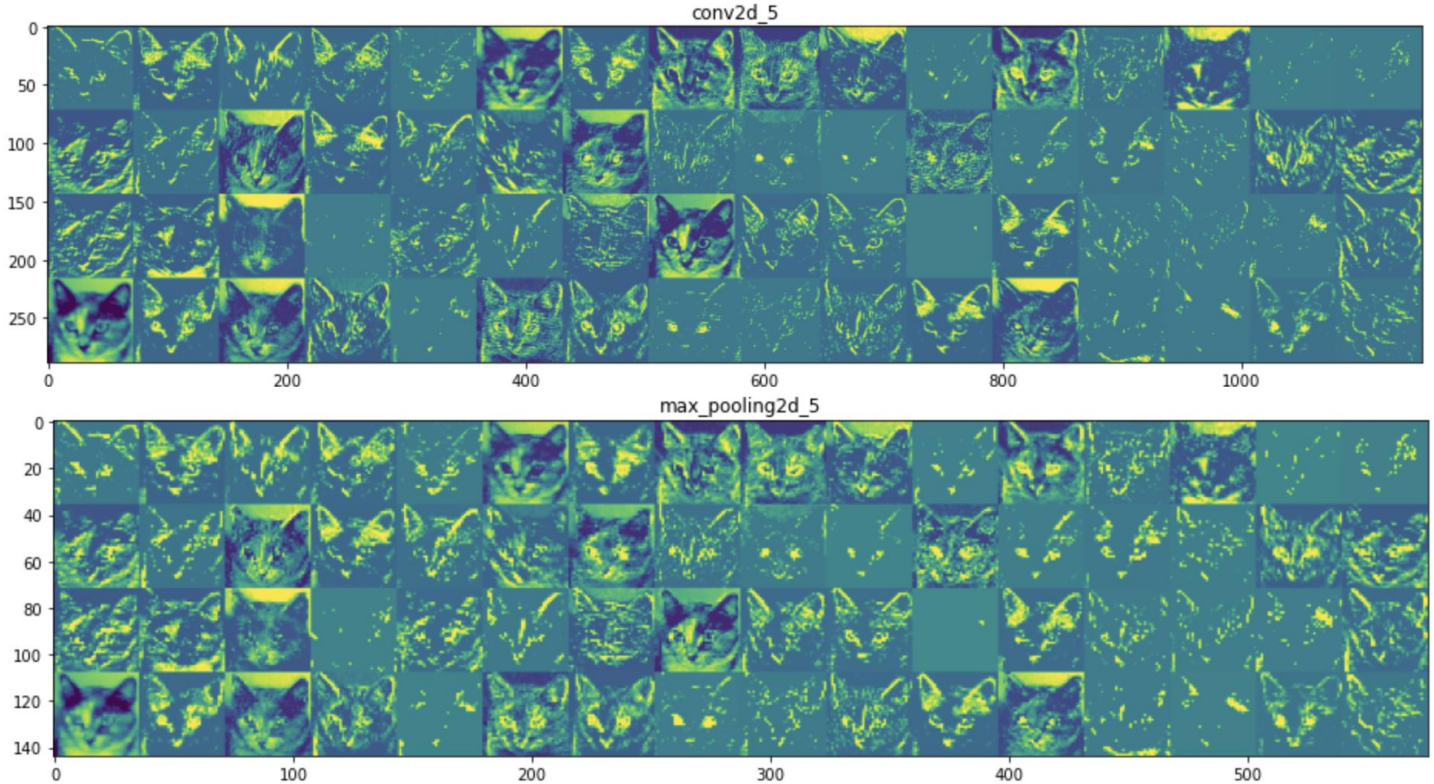


說明：
第一層(conv2d_4)是各種邊緣偵測器的集合，此階段，啟動函數的輸出幾乎保留了初始影像中的所有資訊

5-4-1 中間層輸出的視覺化

✓ (續)視覺化(繪製)每個啟動函數輸出中的每個channel的響應圖

Out[] :



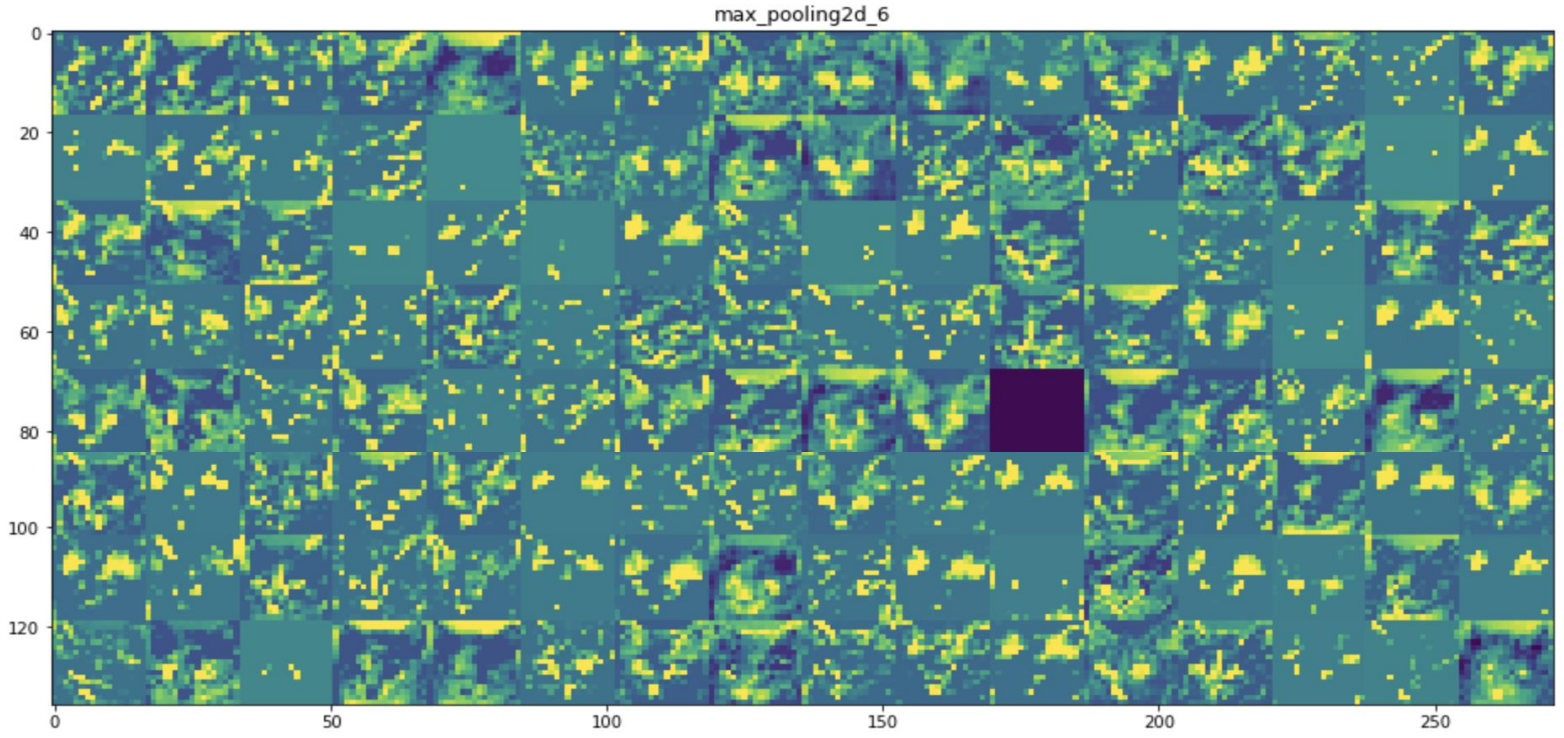
✓ 層級越高，啟動函數的輸出變得越抽象，視覺上也越難解釋

✓ 模型編碼出更高階的概念，如：貓耳朵、貓眼

5-4-1 中間層輸出的視覺化

✓ (續)視覺化(繪製)每個啟動函數輸出中的每個channel的響應圖

Out[] :



✓ 高階內容呈現出較少關於影像的視覺內容資訊，較多關於影像是哪個類別（哪個東西）
✓ 啟動函數輸出的稀疏性隨著層的深度而增加
✓ 越後面filter值是空的(全黑部分)，表示已找不到過濾器要編碼的圖案patten

5-4-1 中間層輸出的視覺化

- ✓ 此過程如同人類一樣，在觀察一個場景幾秒後，我們可以記住場景中的**抽象事物**
- ✓ 如果試圖從記憶中畫出一般的腳踏車，即使看過數以千計的腳踏車，我們仍無法正確地描繪出來，**大腦已經學會將視覺輸入完全抽象化，會將其轉換成高階的視覺概念**，同時過濾掉不相關的視覺細節，使得我們不容易完整記住周圍的事物！



5-4-2 視覺化convnet的過濾器(filter)



- ✓ 另一個視覺化convnet的方法：過濾器(filter)
- ✓ 看各filter要過濾的視覺圖案
  - 透過改變輸入影像的像素，讓filter的響應最大化(梯度上升)來完成
- ✓ 方法
  - 給convnet一張空白的影像
  - 將梯度下降法套用到convnet，直到所指定的層、filter對輸入影像的響應最大化
  - 得到我們想看到的filter影像

補充：

- ✓ **Filter如何看？**
  - 看filter對輸入pattern的響應
- ✓ **響應是一種行為，如何看？**
  - 觀察pattern的圖案
  - 看能對filter產生最大響應的那個pattern長怎樣，來作為filter的長相

5-4-2 視覺化convnet的filter



✓建立過濾器視覺化的損失函數張量

```
from tensorflow.keras.applications import VGG16
from tensorflow.keras import backend as K #載入keras後端backend，用後端k.mean方法來操作張量取其平均值
import tensorflow as tf
tf.compat.v1.disable_eager_execution()
model = VGG16(weights='imagenet', include_top=False) #卷積基底
layer_name = 'block3_conv1' #神經網路層
filter_index = 0 #啟動函數中過濾器值0
layer_output = model.get_layer(layer_name).output
loss = K.mean(layer_output[:, :, :, filter_index]) # 定義損失函數張量, 其為層輸出張量數值取平均
```

補充：

- Keras模型級函式庫避免了繁雜的底層操作，如張量乘積、卷積，這些皆由後端引擎處理
- 目前keras後端有tensorflow、Theano、CNTK
- 若需調整後端功能進行張量操作，需匯入後端模組backend

5-4-2 視覺化convnet的filter



✓ 取得損失值(loss)相對於輸入(model.input)的梯度

```
# gradients() 會傳回一個由張量組成的list, 在本例中, list 的大小為 1, 因此, 只取出其第 0 個元素, 即 grads 是 1 個梯度張量
grads = K.gradients(loss, model.input)[0]
```

✓ 梯度正規化技巧(確保對輸入影像的像素值更新幅度能限制在規範範圍內)

```
# 在做除法之前先加上 1e-5 以避免意外地除以 0
grads /= (K.sqrt(K.mean(K.square(grads))) + 1e-5) #梯度張量平方的平均值在取平方根
```

✓ 建立給定輸入張量，取得輸出張量的keras function

```
iterate = K.function([model.input], [loss, grads]) # 定義一個 Keras 後端函式

import numpy as np
loss_value, grads_value = iterate([np.zeros((1, 150, 150, 3))]) #輸出損失張量與梯度張量
```

輸入numpy張量(大小為1的張量list)

5-4-2 視覺化convnet的filter



✓ 透過隨機梯度下降實作損失最大化

```
# 從帶有雜訊的灰階圖像開始
input_img_data = np.random.random((1, 150, 150, 3)) * 20 + 128.

step = 1. # 每個梯度更新的大小
for i in range(40): # 執行梯度上升 40 步
    loss_value, grads_value = iterate([input_img_data]) # 計算損失值和梯度值
    input_img_data += grads_value * step  # 以朝向最大化損失調整輸入圖像 (以前SGD 是用 -= 算符, 現在反過來是用 += 算符)
```

說明：

目前得到圖像張量

shape(1,150,150,3)的浮點張量，其元素值可能不是[0,255]內的整數，因此需要對此張量進行後處理(postprocession)，以將其轉換成可顯示的影像格式

5-4-2 視覺化convnet的filter



✓ 建立將圖像張量轉換為可用的影像格式的自訂函式

```
def deprocess_image(x):
    x -= x.mean()
    x /= (x.std() + 1e-5)
    x *= 0.1

    x += 0.5
    x = np.clip(x, 0, 1) # 修正成 [0, 1], 即 0-1 之間

    x *= 255
    x = np.clip(x, 0, 255).astype('uint8')
    return x
```

張量正規化：以 0 為中心, 確保 std 為 0.1

轉換成 RGB 陣列

5-4-2 視覺化convnet的filter

✓ 建立過濾器視覺化函數

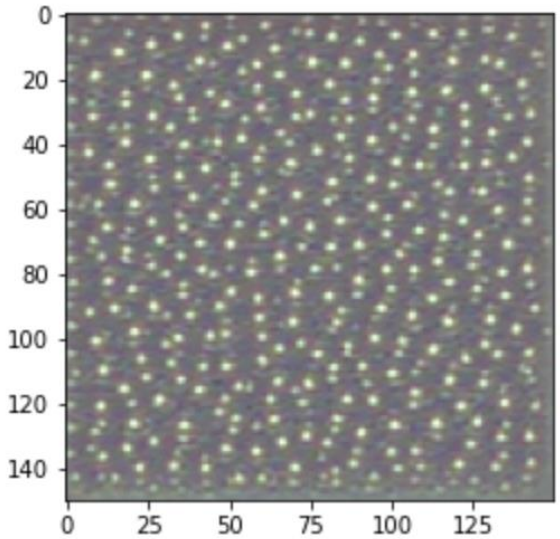
```
def generate_pattern(layer_name, filter_index, size=150):
    layer_output = model.get_layer(layer_name).output  # 取得指定層的輸出張量
    loss = K.mean(layer_output[:, :, :, filter_index])
    # 1.取得指定過濾器的輸出張量, 並以最大化此張量的均值做為損失
    grads = K.gradients(loss, model.input)[0]  # 根據此損失計算輸入影像的梯度
    grads /= (K.sqrt(K.mean(K.square(grads))) + 1e-5) # 標準化技巧：梯度標準化
    iterate = K.function([model.input], [loss, grads])
    # 2.建立 Keras function 來針對給定的輸入影像回傳損失和梯度
    input_img_data = np.random.random((1, size, size, 3)) * 20 + 128.
    # 3. 從帶有雜訊的灰階影像開始
    step = 1.
    for i in range(40): # 執行梯度上升 40 步
        loss_value, grads_value = iterate([input_img_data]) # 4. 針對給定的輸入影像回傳損失和梯度
        input_img_data += grads_value * step
    img = input_img_data[0]
    return deprocess_image(img) # 進行圖像後處理後回傳
```

5-4-2 視覺化convnet的filter

✓ (續)建立過濾器視覺化函數

```
plt.imshow(generate_pattern('block3_conv1', 0))
# 我們來看看 block3_conv1 層中的過濾器 0 的特徵圖
```

Out[] :



說明：block3_conv1 層中的第0個channel最大的回應pattern

5-4-2 視覺化convnet的filter



✓ 產生一層中所有過濾器響應pattern

```
for layer_name in ['block1_conv1', 'block2_conv1', 'block3_conv1', 'block4_conv1’]:
    size = 64
    margin = 5

    # 1. 用於儲存結果的空(黑色)影像
    results = np.zeros((8 * size + 7 * margin, 8 * size + 7 * margin, 3))

    for i in range(8): # ← 迭代產生網格的行
        for j in range(8): # ←迭代產生網格的列
        # 在 layer_name 中產生過濾器 i +(j * 8) 的 pattern
        filter_img = generate_pattern(layer_name, i + (j * 8), size=size) #只看每層中前64個過濾器
```

5-4-2 視覺化convnet的filter



✓ (續)產生一層中所有過濾器響應pattern

```
# 將結果放在結果網格的方形(i, j)中
horizontal_start = i * size + i * margin
horizontal_end = horizontal_start + size
vertical_start = j * size + j * margin
vertical_end = vertical_start + size
results[horizontal_start: horizontal_end, vertical_start: vertical_end, :] = filter_img
```

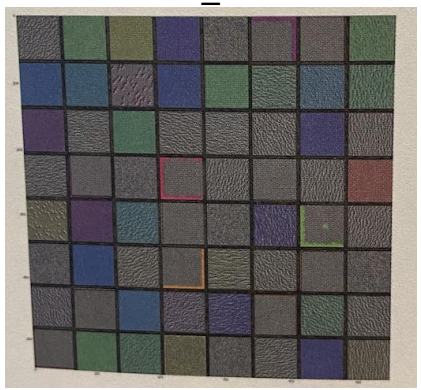
5-4-2 視覺化convnet的filter

✓ (續)產生一層中所有過濾器響應pattern

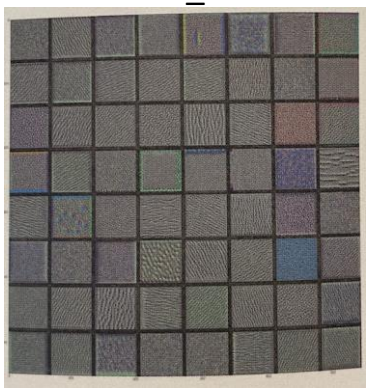
```
# 顯示網格結果
plt.figure(figsize=(20, 20))
plt.imshow(results)
plt.show()
```

Out[] :

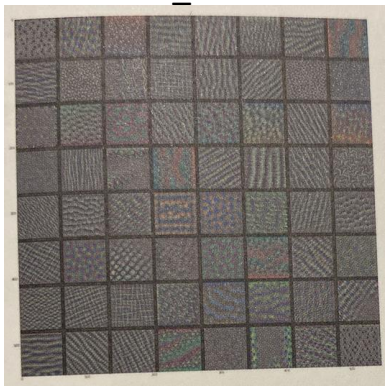
'block1_conv1



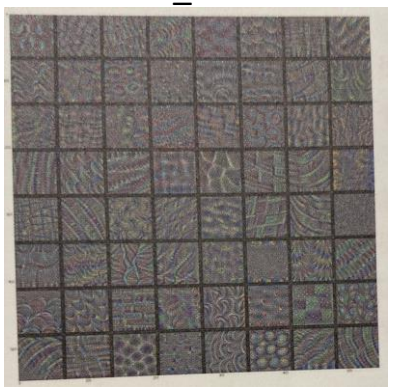
'block2_conv1



'block3_conv1



'block4_conv1



說明：較高層中的過濾器，會越來越接近自然影像中的紋理：羽毛、眼睛、樹葉等

5-4-3 視覺化類別激活熱圖

- ✓ 類別激活熱圖 (class activation map, CAM)
  - 視覺化技術
  - 幫助理解影像中哪些部分會讓convnet做出最終分類的決策，在分類錯誤的情況下更有用
  - 一個2D的分數網格圖
  - 針對輸入影像的每個位置進行計算，指示出每個位置相對於目前類別的重要性
  - 如：將影像餵入狗V.S.貓的神經網路中
    - ✓ CAM對貓及狗類別產生熱圖
    - ✓ 顯示出各部分像貓的程度與各部分像狗的程度

5-4-3 視覺化類別激活熱圖

✓ 載入預先訓練權重的VGG16神經網路

```
from keras.applications.vgg16 import VGG16
model = VGG16(weights='imagenet') # 請注意, 在頂部包含了密集連接的分類器 (預設 include_top=True)
```

5-4-3 視覺化類別激活熱圖

✓ 預先處理VGG16的影像輸入

```
from keras.applications.vgg16 import preprocess_input, decode_predictions
from tensorflow.keras.utils import load_img
from tensorflow.keras.utils import img_to_array
import numpy as np

img_path = r'C:\Users\chche\African_Bush_Elephants.jpg'

img = image.load_img(img_path, target_size=(224, 224))
print(type(img))  # 目前圖片為 <class 'PIL.Image.Image'> 物件
print(img.size)  # 可以用 size 屬性查看尺寸 -> (224, 224)
```

5-4-3 視覺化類別激活熱圖

✓ (續)預先處理VGG16的影像輸入

```
x = image.img_to_array(img) # 將 PIL 物件轉為 float32 的 Numpy 陣列
print(x.shape) # shape=(224, 224, 3)

# 將 x 陣列 (可視為張量) 增加一個批次軸, shape=(1, 224, 224, 3)
x = np.expand_dims(x, axis=0)
print(x.shape)

x = preprocess_input(x) # 預處理批次量 (這會對每一 channel 做顏色值正規化)
```

[Out:]

```
<class 'PIL.Image.Image'>
(224, 224)
(224, 224, 3)
(1, 224, 224, 3)
```

5-4-3 視覺化類別激活熱圖

✓ 使用VGG16神經網路預測圖片類別

```
preds = model.predict(x)
print('預測結果:', decode_predictions(preds, top=3)[0])

np.argmax(preds[0]) #設0因為只有輸入一張圖片
```

[Out:]
[('n02504458', 'African_elephant', 0.8831972), ('n01871265', 'tusker', 0.101020955), ('n02408429', 'water_buffalo', 0.008600049)]
386 #數值最高的元素對應到非洲象類別，其元素索引值為386

補充 :
- 非洲象African_elephant(機率88.3%)
- 有長牙的象tusker(機率10.1%)
- 水牛water_buffalo(機率0.86%)

5-4-3 視覺化類別激活熱圖

✓ 設定Grad-CAM演算法

```
from keras import backend as K
african_elephant_output = model.output[:, 386] # 預測向量中的 "非洲象" 項目
last_conv_layer = model.get_layer('block5_conv3')
# block5_conv3 層的輸出特徵圖, 其為 VGG16 中的最後一個卷積層

grads = K.gradients(african_elephant_output, last_conv_layer.output)[0]
# block5_conv3 的輸出特徵圖中關於 "非洲象" 類別的梯度

pooled_grads = K.mean(grads, axis=(0, 1, 2))
#轉換成向量 shape = (512, ), 其中每個項目是特定特徵圖 channel 的梯度平均強度(值)

#  給定輸入影像的條件下, 讓我們可以存取剛剛定義的數值：pooled_grads 和 block5_conv3 的輸出特徵圖
iterate = K.function([model.input], [pooled_grads, last_conv_layer.output[0]])
```

5-4-3 視覺化類別激活熱圖



✓ (續)設定Grad-CAM演算法

```
# 對於給定的兩隻大象樣本影像, 產生這兩個量值, 以 Numpy 陣列呈現
pooled_grads_value, conv_layer_output_value = iterate([x])

# 將特徵圖陣列中的每個 channel 與 "大象" 類別相關的 "此 channel 的重要程度" 相乘
for i in range(512):
    conv_layer_output_value[:, :, i] *= pooled_grads_value[i]

# 特徵圖的跨 channel 平均值是類別激活函數輸出的熱圖
heatmap = np.mean(conv_layer_output_value, axis=-1)
```

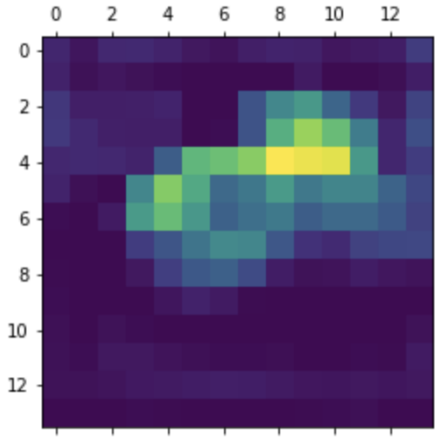
5-4-3 視覺化類別激活熱圖

✓ 熱圖後期處理

```
import matplotlib.pyplot as plt
import numpy as np

heatmap = np.maximum(heatmap, 0)
heatmap /= np.max(heatmap)
plt.matshow(heatmap)
plt.show()
```

Out:



5-4-3 視覺化類別激活熱圖

✓ 將熱圖與原始影像疊加在一起

```
import cv2  #用cv2載入原始影像
img = cv2.imread(img_path)

heatmap = cv2.resize(heatmap, (img.shape[1], img.shape[0])) #將熱圖大小調整為與原始影像大小相同
heatmap = np.uint8(255 * heatmap) #將熱圖轉換為RGB
heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET) #將熱圖疊加到原始影像
superimposed_img = heatmap * 0.4 + img  # 這裡 0.4 是熱圖強度因子
```

5-4-3 視覺化類別激活熱圖

✓ (續)將熱圖與原始影像疊加在一起

```
print('是否儲存成功:', cv2.imwrite('elephant_cam.jpg', superimposed_img))
```

Out:



總結



- ✓ 目前為止，應該知道的內容：
  - 了解卷積神經網路(CNN/convnets)
  - 用資料擴增術來減輕overfitting
  - 使用預先訓練的卷積神經網路做特徵萃取
  - 微調預先訓練的卷積神經網路
  - 視覺化剖析卷積神經網路的學習方式及分類決策